



Institut Africain d'Informatique
Etablissement Inter – Etats d'Enseignement Supérieur
BP 2263 Libreville (Gabon) Tél. (241) 07 70 55 00 - 07 70 56 00
Site web: www.iaisiege.com E-mail: contact@iaisiege.com



Ingénierie Informatique Soft
RCCM-NI-NIA-2012-B-2082 / NIF : 23112
BP: 10084 Niamey Niger/ Site web: www.2isoft.com
E-mail : info@2isoft.com / 2isoft.com@gmail.com

MEMOIRE DE FIN DE CYCLE

En vue de l'obtention du diplôme
D'Ingénieur de Conception en Informatique

Thème

**Conception et mise en œuvre d'un Progiciel de
Gestion intégré pour la gestion interne de
2iSoft**

Présenté et soutenu par

ALI TASSIOU Abass

Superviseur

M. MATY Maman
Enseignant permanent à IAI

Maître de stage

M. HAMZA ISSA Faroukou
Directeur Général de 2iSoft

ANNEE ACADEMIQUE : 2014-2015

DEDICACES

A ma défunte mère Assiatou Elhadj Souleymane et mon père Ali Tassiou ;

A mes tantes (Mariama Gambo et Mariama Alio Sarki) pour leurs soutiens, ainsi que leurs encouragements durant ces trois années ;

A toute ma famille.

Je leur dédie ce travail.

REMERCIEMENTS

Je commence par remercier « ALLAH » qui m'a donné la force et la santé pour mener à bien ce travail.

La rédaction de ce mémoire n'aurait pas été possible sans l'aide d'un grand nombre de personnes. Je souhaite ici les en remercier.

Je tiens à exprimer ma reconnaissance et mon remerciement à mon maître de stage M. HAMZA ISSA Faroukou et mon superviseur M. MATY Maman qui ont été d'un grand apport dans la réalisation de ce travail.

Mes remerciements vont également :

- ✓ Au personnel de 2isoft qui n'a ménagé aucun effort pour m'accompagner dans la réalisation de ce travail ;
- ✓ Au corps professoral de l'IAI pour les enseignements prodigués ;
- ✓ A l'Association Karfafa Zoumountchi (AKZ) ;
- ✓ A la grande famille des Anciens Enfants de Troupe (AET) ;
- ✓ A l'Association des Musulmans de l'IAI ;
- ✓ A la communauté estudiantine de l'IAI ;
- ✓ Aux membres du jury pour l'intérêt qu'ils ont porté à mes recherches en acceptant d'examiner mon travail et de l'enrichir par leurs propositions ;
- ✓ A tous ceux qui de près ou de loin ont contribué à la réussite de ma formation.



Avant-propos

L'Institut Africain d'Informatique (I.A.I) est une École Inter États d'Enseignement Supérieur créée en 1971 à Fort-Lamy au Tchad. Il regroupe onze(11) États Africains. Il intègre dans le cursus de formation des ingénieurs de conception en informatique, en fin de cycle, un stage de formation pratique d'une durée de 5 (cinq) mois en entreprise ou en centre de recherche. Ce stage vise à mettre les élèves ingénieurs dans un environnement de conception d'applications informatiques ou dans un centre de recherche et de développement (laboratoire) afin de leur permettre une intégration en milieu professionnel et/ou scientifique.

Le présent mémoire rentre dans le cadre de l'obtention du diplôme de fin d'étude du cycle ingénieur de l'Institut Africain d'Informatique (IAI). Il présente le travail que j'ai effectué durant cinq (5) mois au sein de 2isoft.

RESUME

Au fur et à mesure qu'une entreprise se développe, elle manifeste le besoin d'outils performants et efficaces lui permettant de mieux gérer ses activités. Le choix d'une solution informatique doit se faire en tenant compte non seulement du coût de la solution, mais aussi des opportunités qu'offre cette dernière et des contraintes qu'elle implique.

Dans le souci d'améliorer sa productivité et la performance globale de son système, 2iSoft a entrepris le projet consistant à réunir toutes les activités de l'entreprise autour d'une seule et unique base de données. Vu que les activités à automatiser pourraient évoluer avec le temps, il faudrait alors que le nouveau système à développer soit modulaire ; d'où le choix d'une solution ERP, car la modularité et l'utilisation d'une base de données unique sont les principaux éléments qui définissent un ERP. L'idéal serait un développement sur mesure car les logiciels sont censés supporter les processus métiers et non l'inverse. Chaque éditeur a ses propres motivations, ses propres contraintes qui font qu'une solution ERP existante déjà ne lui convienne pas. C'est dans ce cadre qu'il nous a été demandé de concevoir et de mettre en œuvre un progiciel de gestion intégré pour gérer les activités de 2iSoft.

Ce progiciel permettra de gérer les achats et les ventes des produits et services ; gérer le paiement des taxes et cotisations aux établissements financiers ; gérer la trésorerie ; éditer les bons de commande, les factures clients et les bons de livraison ; gérer les tâches ; gérer les employés ; gérer les clients et les fournisseurs.

Pour mener à bien notre travail nous avons eu à utiliser SCRUM comme canevas pour l'application des procédés et techniques de développement. Dans la mise en œuvre des sprints nous avons utilisé un processus UP

Mots clés : ERP, PGI, SCRUM, Processus UP.

ABSTRACT

Gradually a company grows, it demonstrates the need for effective and efficient tools to better manage its activities. Choosing an IT solution must take into account not only the cost of the solution, but also the opportunities of the latter and the constraints it entails.

In order to improve productivity and overall performance of its system, 2iSoft undertook the project of bringing together all the company's activities around a single database. Given that automate activities could evolve over time, it would then have to develop the new system is modular; hence the choice of an ERP solution as modularity and the use of a single database are the main elements that define an ERP. Ideally, a custom software development because are expected to bear the business processes and not the reverse. Each publisher has its own motivations, constraints that make an already existing ERP solution does not suit him. It is in this context that we were asked to design and implement an ERP system to manage the activities of 2iSoft.

This software will manage purchases and sales of products and services; manage the payment of taxes and fees to financial institutions; cash management; edit orders, customer invoices and delivery notes; manage tasks; manage employees; manage customers and suppliers.

To carry out our work we had to use Scrum as a framework for the implementation of development processes and techniques. In the implementation of sprints we used an unified process.

Keywords: ERP, SCRUM, Unified Process.

TABLE DES MATIERES

DEDICACES.....	i
REMERCIEMENTS	ii
Avant-propos	iii
RESUME.....	iv
ABSTRACT	v
TABLE DES MATIERES.....	vi
LISTE DES FIGURES	ix
LISTE DES TABLEAUX	x
INTRODUCTION GENERALE.....	1
PREMIERE PARTIE : CONTEXTE GENERAL DE L'ETUDE	2
Chapitre I : STRUCTURE D'ACCUEIL ET SUJET D'ETUDE	3
Introduction.....	3
I.1 Présentation de 2iSoft.....	3
I.1.1 Localisation.....	3
I.1.2 Missions de 2iSoft.....	3
I.1.3 Organigramme de 2iSoft	3
I.2 Présentation du sujet	4
I.2.1 Libellé du sujet.....	4
I.2.2 Problématique.....	4
I.2.3 Objectif du projet	4
I.2.4 Résultats attendus.....	4
Conclusion	5
Chapitre II : ETUDE GENERALE DU SUJET	6
Introduction.....	6
II.1 Définition de l'ERP	6
II.2 Présentation de l'ERP	7
II.3 Typologie d'ERP	7
II.4 Les bénéfices d'un ERP pour l'entreprise sont les suivants :	9
II.5 Critère de sélection d'un ERP	9
II.6 Quel est le périmètre d'un ERP ?.....	9
II.7 L'ERP peut-il s'adapter aux règles de gestion de mon entreprise ?	10
II.8 Architecture technique d'un ERP	10
II.9 Les avantages et inconvénients d'un ERP	11
Conclusion	12

DEUXIEME PARTIE : ANALYSE ET CONCEPTION	13
Chapitre III : CHOIX DE LA METHODE D'ANALYSE ET DE CONCEPTION	14
Introduction.....	14
III.1 Les méthodes d'analyse et de conception	14
III.2 Choix d'une méthode	15
III.3 Pourquoi adopter les méthodes agiles ?	15
III.4 Présentation de Scrum ⁽⁴⁾	17
III.5 Présentation du processus unifié (UP)	23
III.6 Présentation d'UML (Unified Modified Language).....	25
Conclusion	26
Chapitre IV : ANALYSE ET DE CONCEPTION.....	27
Introduction.....	27
IV.1 Etude de l'existant.....	27
IV.1.1 Description de l'existant.....	27
IV.1.2 Critique de l'existant	27
Perspective	27
IV.2 Déroulement des sprints.....	27
IV.2.1 Sprint 0	27
IV.2.2 : Construction du cœur de notre système	31
IV.2.3 Sprint 1 : gestion des utilisateurs	35
IV.2.4 Sprint 2 : Eléments de base	38
IV.2.5 Sprint 3 : Gestion des Achats	42
IV.2.6 Sprint 4 : Gestion des ventes.....	48
IV.2.7 Sprint 5 : Gestion des tâches.....	54
Conclusion	57
TROISIEME PARTIE : REALISATION ET CONDUITE DE PROJET.....	58
Chapitre V : REALISATION.....	59
Introduction.....	59
V.1 Les outils utilisés	59
V.2 Déploiement de la plateforme web	62
V.3 Architecture réseau de la solution	62
V.4 Menaces et risques applicatifs.....	63
V.5 Captures d'écran	66
Conclusion	68
Chapitre VI : Conduite de projet	69
Introduction.....	69

VI.1 Intervenants au projet	69
VI.2 Estimation du cout de réalisation	69
VI.3 Planification des tâches.....	70
VI.4 Diagramme de Gantt prévisionnel	71
VI.5 Exécution des tâches.....	72
VI.6 Diagramme de Gantt réel.....	73
Conclusion	73
Conclusion générale	74
REFERENCES BIBLIOGRAPHIQUES	75
Ouvrage	75
Sites web consultés	75

LISTE DES FIGURES

Figure 1: Organigramme 2isoft	3
Figure 2: Présentation ERP	7
Figure 3: Architecture technique d'un ERP	11
Figure 4: Présentation Scrum	17
Figure 5: Phase du processus unifié	24
Figure 6: Diagramme de contexte statique	29
Figure 7: Fonctionnement du système	31
Figure 8: Cas d'utilisation gestion des utilisateurs	35
Figure 9: Diagramme de classe, gestion des utilisateurs	38
Figure 10: Diagramme des cas d'utilisation, Eléments de base	39
Figure 11: Diagramme de séquence, enregistrer client	40
Figure 12: Diagramme de séquence enregistrer fournisseur	41
Figure 13: Diagramme de séquence, enregistrer produit	42
Figure 14: Diagramme des cas d'utilisation, gestion des achats	43
Figure 15: Diagramme de séquence, Enregistrer commande fournisseur	44
Figure 16: Diagramme de séquence, règlement fournisseur	46
Figure 17: Diagramme de classe, gestion des achats	47
Figure 18: Diagramme des cas d'utilisation, gestion des ventes	48
Figure 19: Diagramme de séquence, enregistrer commande client	49
Figure 20: Diagramme de séquence, enregistrer livraison client	51
Figure 21: Diagramme de séquence, enregistrer règlement client	52
Figure 22: Diagramme de classe, gestion des ventes	53
Figure 23: Diagramme des cas d'utilisation, gestion des tâches	54
Figure 24: Diagramme de classe, gestion des tâches	57
Figure 25: Architecture 3 tiers	62
Figure 26: Architecture réseau de la solution	62

LISTE DES TABLEAUX

Tableau 1:Product backlog	30
Tableau 2: Sprint backlog, gestion des utilisateurs	35
Tableau 3: Sprint backlog, Eléments de base	39
Tableau 4: Sprint backlog, gestion des achats	42
Tableau 5: Sprint backlog, gestion des ventes	48
Tableau 6: Sprint backlog, gestion des tâches	54
Tableau 7: Correspondances entre les définitions de l'OWASP et du WASC.....	65
Tableau 8: Intervenants au projet.....	69
Tableau 9: coût de réalisation	69
Tableau 10: Planification des tâches	70
Tableau 10: Exécution des tâches	72

INTRODUCTION GENERALE

L'accès des entreprises aux nouvelles technologies, et à l'internet en particulier, tend à modifier la communication entre les différents acteurs du monde des affaires. Notamment entre l'entreprise et ses clients, le fonctionnement interne de l'entreprise et la relation de l'entreprise avec ses différents partenaires et fournisseurs.

2iSoft à l'instar de beaucoup d'entreprises collecte, génère et accumule de grandes quantités de données. Ces données sont gérées par des systèmes disjoints ; d'où une fragmentation de l'information au sein de la société. Cette situation freine la productivité et la performance globale de l'entreprise.

C'est dans ce cadre que la société d'Ingénierie Informatique Soft (2iSoft) nous a confié la tâche consistant à concevoir et mettre en œuvre un progiciel de gestion intégré pour gérer ses activités.

Notre travail consistera donc à assurer une gestion unique, cohérente et sécurisée des données de l'entreprise en mettant fin aux problèmes d'interfaçage, de synchronisation et de doubles saisies.

Pour mener à bien ce travail, nous l'avons scindé en trois (3) grandes parties :

- La première partie sera consacrée au contexte général de l'étude. Nous y présenterons la structure qui nous a accueillis pour cinq mois de stage, le sujet, ses contours et les concepts liés au domaine d'étude.
- La deuxième partie abordera l'analyse et la conception du système. Nous y présenterons un certain nombre de méthodes de développement, avant de choisir la méthode qui sera utilisée pour la réalisation de notre projet.
- La troisième partie traitera de la réalisation et la conduite de projet.

PREMIERE PARTIE : CONTEXTE GENERAL DE L'ETUDE

Cette partie a pour objectif de présenter le cadre dans lequel nous avons effectué notre stage. C'est ainsi qu'après avoir présenté la structure d'accueil et le sujet, nous présenterons les concepts clés nécessaires à la compréhension de ce dernier.

Chapitre I : STRUCTURE D'ACCUEIL ET SUJET D'ETUDE

Introduction

Nous avons effectué notre stage au sein de la société d'Ingénierie Informatique Soft (**2iSoft**). 2iSoft est une société informatique agissant dans le domaine de conception des logiciels de gestion et accessoirement accompagne ses clients par les formations et la prestation dans tout autre domaine informatique. 2iSoft vous offre des services de qualité en conseils, des solutions clé en main dans le domaine des NTIC.

I.1 Présentation de 2iSoft

I.1.1 Localisation

2iSoft est située à l'immeuble ex radio Souda, rue du Nigeria, quartier nouveau marché (Niamey-Niger). Elle a un riche potentiel humain composé de collaborateurs, d'ingénieurs permanents et consultants, et des techniciens supérieurs. **2iSoft** est dirigée par Monsieur **HAMZA ISSA Faroukou**.

I.1.2 Missions de 2iSoft

Grâce aux importantes ressources matérielles et humaines dont elle dispose, **2iSoft** offre une multitude de services informatiques. Nous citerons entre autres :

- Développement de logiciels
- Conception de sites web
- Graphisme et infographie
- Formation et conseils en informatique
- Maintenance et réseau informatique
- Vente et fourniture de matériels informatiques

I.1.3 Organigramme de 2iSoft

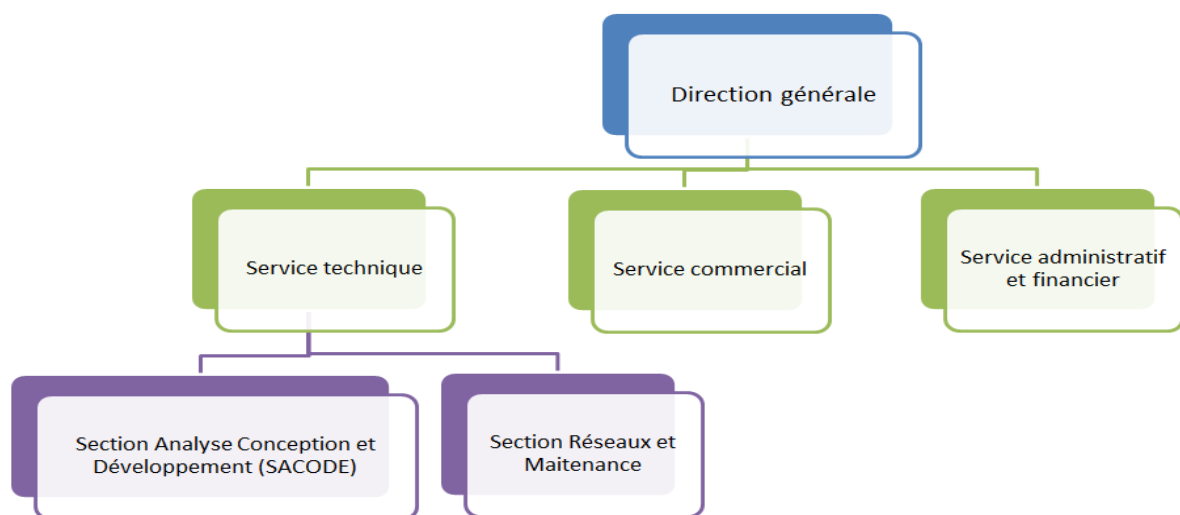


Figure 1: Organigramme 2isoft

I.2 Présentation du sujet

I.2.1 Libellé du sujet

« Conception et mise en œuvre d'un progiciel de gestion intégré pour la gestion interne de 2iSoft ».

I.2.2 Problématique

2isoft voudrait développer un système d'information complètement intégré, c'est adire un système dans lequel chaque élément d'information entrerait une seule fois afin d'accroître les performances de l'entreprise en permettant notamment d'éviter dans une certaine mesure des dysfonctionnements. Elle mettra donc en réseau, à travers une base de données unique, l'ensemble des données relatives aux fonctions de l'entreprise (achat, vente, stock, personnel, comptabilité, etc.)

C'est dans cette perspective d'amélioration de la performance de l'entreprise que nous nous posons les questions suivantes :

- Comment mettre en place un système permettant de gérer toutes les activités de l'entreprise ?
- Comment assurer la cohérence et l'homogénéité des informations ?
- Comment assurer la maintenabilité et l'évolutivité du système ?
- Quelle politique de sécurité adopter pour ne permettre l'accès qu'aux usagers autorisés et assurer la confidentialité des données ?
- Quelle méthodologie adopter pour mener à bien notre travail ?

I.2.3 Objectif du projet

L'objectif de ce projet est de fournir à 2iSoft un système réalisé selon les critères de la société et en fonction des besoins spécifiques de celle-ci. Ce système doit :

- Assurer la cohérence et l'homogénéité des informations ;
- Assurer l'intégrité et l'unicité du système d'information ;
- Faciliter la communication interne et externe ;
- Accroître la productivité de l'entreprise.

I.2.4 Résultats attendus

A l'issue de notre travail, les résultats suivants sont attendus :

- Sur le plan fonctionnel
 - Gestion des achats ;
 - Gestion des ventes ;
 - Gestion comptable ;
 - Contrôle de gestion (Possibilité de bloquer les utilisateurs non autorisés, Traçabilité des opérations) ;
 - Gestion des stocks (logistique) ;
 - Gestion des tâches ;
 - Gestion des ressources humaines.

➤ Sur le plan technique

- Convivialité et simplicité d'utilisation du logiciel ;
- Un bon niveau de paramétrage du logiciel ;
- Construire l'application sous forme de modules indépendants mais parfaitement compatibles sur une base de données unique et commune

Conclusion

Ce chapitre nous a permis de présenter le centre d'accueil, puis de faire le point sur le travail qui nous a été demandé.

Chapitre II : ETUDE GENERALE DU SUJET

Introduction

Dans ce chapitre, nous allons faire une présentation générale des concepts liés à notre domaine d'étude. Nous ferons d'abord une étude de l'ERP, puis nous verrons les différents éléments clés nécessaires à la réalisation d'un ERP.

II.1 Définition de l'ERP

La définition ERP vient de l'anglais « Entreprise Ressource Planning ». ERP a été traduit en français par l'acronyme PGI (Progiciel de Gestion intégré) et se définit comme un groupe de module relié à une base de données unique. Un ERP est donc :

Un progiciel : application développée par un éditeur, suffisamment générale pour répondre aux besoins de plusieurs clients. Il ne s'agit donc pas d'un logiciel spécifique maison développé par une entreprise. Il comprend en fait une base standard et une partie personnalisable à travers un paramétrage⁽¹⁾.

Une application de gestion, conçue en premier lieu pour automatiser les transactions administratives de l'entreprise : comptabilité, gestion des stocks, suivis des commandes et du programme de production,... Un ERP permet de saisir les transactions et propage l'information recueillie vers les niveaux pertinents. Toutefois, il ne contient pas de programme d'optimisation ou de décision automatique⁽¹⁾.

Un produit intégré, c'est à dire qu'il prend en compte l'ensemble des fonctions processus de l'entreprise de manière intégrée et automatisée. Il est architecturé de sorte à assurer une gestion unique, cohérente et sécurisée des données en temps réel : il garantit à tout instant une intégrité et une cohérence parfaite des données pour tous les utilisateurs. Cette technologie met donc fin aux problèmes d'interfaçage, de synchronisation et de double saisie⁽¹⁾.

Il s'agit donc d'une application informatique formée de modules fonctionnels standards, reliés directement à une base de données unique et couvrant l'ensemble des processus de l'entreprise. Un ERP constitue par ailleurs le plus souvent une solution de dimension internationale capable de gérer des contextes multi-législations, multi-langues, multidevises ; il permet ainsi la remontée des informations émanant des filiales d'un groupe dans différents pays. Cette caractéristique peut sembler anodine, mais elle est fondamentale à l'heure de la mondialisation car l'environnement linguistique et légal est un levier structurant pour une entreprise.

II.2 Présentation de l'ERP

L'ERP est un progiciel qui permet de gérer l'ensemble des processus opérationnels d'une entreprise, en intégrant plusieurs fonctions de gestion : solution de gestion des commandes, solution de gestion des stocks, solution de gestion de paie et de la comptabilité, etc. dans un système.

Pour être qualifiée de « progiciel de gestion intégré », une solution logicielle ERP doit couvrir au moins deux principes fondamentaux qui sont les suivants :

- construire des applications informatiques sous forme de modules indépendants mais parfaitement compatibles sur une base de données unique et commune.
- L'usage d'un moteur de workflow permet de définir l'ensemble des tâches d'un processus et de gérer leur réalisation dans tous les modules du système qui en ont besoin.



Figure 2: Présentation ERP

II.3 Typologie d'ERP

Depuis quelques années deux concepts pour les ERP ont émergé : Les ERP verticaux et les horizontaux. Les ERP verticaux sont des ERP qui sont spécifiques à des métiers comme le bâtiment, cabinet médical, etc. et les horizontaux sont des ERP qui traitent des fonctions qui sont plus génériques.

II.3.1 ERP pour les TPE/TPI

A. ERP Horizontal : Booster Pro

Il a été créé par l'éditeur Consoma pour les petites entreprises de 1 à 50 personnes. Booster Pro ERP est architecturé autour de différents modules : Clients, Répertoire, Produits, Fournisseurs, Commandes, Statistiques, écritures comptables, etc. Booster Pro centralise toutes les données de votre activité, pour les mettre à disposition de vos utilisateurs en

fonction de leur profil, avec toutes les fonctions de sécurité nécessaires au partage d'informations sensibles.

Le logiciel possède un tableau de bord qui permet de suivre en temps réel la bonne marche de votre entreprise. Selon son éditeur, il est capable de répondre au besoin de structures ayant jusqu'à 50 employés.

B. ERP Vertical : Manufacturing PAC

Cet ERP qui est plus orienté métier a été créé par l'éditeur Cegid manufacturing. Les moyens financiers et humains d'une très petite entreprise sont assez limités. Cet ERP est parfaitement adaptée à ces contraintes et permet donc une gestion optimale. Il est composé des modules de vente, d'achat, de stock, de production et de comptabilité. Il permet de générer des tableaux de bord précis et de faire du reporting.

II.3.2 ERP pour les PME/PMI

A. ERP horizontal: Gestimum ERP PME

Progiciel élaboré par l'éditeur Gestimum. Cet ERP optimise l'ensemble des processus opérationnels de la PME : gestion comptable, gestion commerciale, CRM, ventes, reporting (temps réel), achats, stocks, planning.

Les principales composantes fonctionnelles de l'entreprise sont intégrées dans ce progiciel.

B. ERP Vertical : IXBat

Progiciel conçu par l'éditeur CGSI pour répondre aux besoins de gestion des entreprises spécialisés dans des métiers de production comme le Bâtiment et les Travaux Publics. iXBAT est un système complet ERP BTP et parfaitement intégré. Il apporte une aide efficace aux entreprises qui visent l'efficacité dans le pilotage de leur activité et de leur gestion.

Il offre une gestion des chantiers, une gestion du parc matériel, une gestion financière (comptabilité et tableaux de synthèse ...), gestion des ressources humaines et il donne en temps réel à l'entreprise une visibilité sur la rentabilité de ses chantiers avec du reporting.

II.3.3 Chez les grands groupes

SuccessFactor Business Execution Suite est un progiciel de l'éditeur SAP présenté comme possédant les fonctionnalités suivantes :

- Finance (Financial Management)
- Business Intelligence (Human Resource Management, Manufacturing Resource Planning)
- Recrutement (Human Resource Management)
- Suivi de projet (Human Resource Management)
- Analyse des performances (Human Resource Management)
- Planification (Supply Chain Management)
- Communication (CRM, Human Resource Management)

Ce progiciel possède donc toutes les caractéristiques requises pour bénéficier du statut d'ERP. Il est déployé entre autres chez Nissan qui emploie quelques 215 000 salariés.

II.4 Les bénéfices d'un ERP pour l'entreprise sont les suivants :

- Optimisation des processus de gestion (flux économiques et financiers) ;
- Cohérence et homogénéité des informations ;
- Intégrité et unicité du système d'information ;
- Partage du même système d'information facilitant la communication interne et externe ;
- Minimisation des coûts de gestion de l'information ;
- Globalisation de la formation (même logique, même ergonomie) ;
- Productivité accrue ;
- Maîtrise des coûts, des délais de mise en œuvre et de déploiement ⁽²⁾.

II.5 Critère de sélection d'un ERP

Voici quelques critères qui aideront les décideurs IT dans leur choix de solution ERP :

- Pérennité
 - Du progiciel ;
 - De l'intégrateur et de l'éditeur ;
 - De la plateforme.
- Utilisabilité
 - Simple ;
 - Efficace ;
 - Intuitif ;
 - Ergonomique.
- Fonctionnalités
 - Domaines fonctionnels couverts ;
 - Intégration des outils décisionnels ;
 - Niveau de paramétrages.
- Sécurité des données
 - Possibilité de bloquer les utilisateurs non autorisés ;
 - Possibilité d'empêcher la saisie accidentelle d'informations incorrectes ;
 - Traçabilité des modifications.
- Maîtrise des flux : management de la performance
 - Aide à la décision ;
 - Intégrité de la base de données ;
 - Publication et restitution des données.
- Aspect international
 - Le logiciel est-il adapté aux dispositions légales des pays où l'entreprise est implantée ?
 - L'interface est-elle disponible dans d'autre langue ?

II.6 Quel est le périmètre d'un ERP ?

Un progiciel de gestion intégré permet de construire un système d'information homogène sur une base de données unique. Il couvre alors un large périmètre de gestion :

- gestion des achats
- gestion des ventes
- gestion comptable
- contrôle de gestion
- gestion des stocks (logistique)

Ainsi l'ERP est constitué de plusieurs modules correspondant chacun à un des domaines de gestion et garantit l'unicité des informations qu'il contient puisqu'il n'a qu'une seule base de données au sens logique.

II.7 L'ERP peut-il s'adapter aux règles de gestion de mon entreprise ?

Les ERP ont des fonctions standards qui s'adaptent aux principales règles de gestion des entreprises (achats, ventes, logistique, ...)

Cependant, chaque entreprise est unique et les ERP ne peuvent pas s'adapter en standard à tous les cas particuliers d'entreprises.

Deux solutions sont alors envisageables :

- Réaliser des développements spécifique, souvent coûteux et techniquement risqués car pouvant impacter tous les modules
- Revoir les règles de gestion spécifiques pour les adapter au progiciel. Cette démarche implique une analyse préalable des processus et une réflexion à mener par la maîtrise d'ouvrage puis une bonne conduite du changement, lors de la mise en place du progiciel. En effet, les employés doivent être préparés à voir la nature de leur travail évoluer, à adopter de nouvelles règles de gestion et à abandonner les anciens logiciels qu'ils utilisaient.

II.8 Architecture technique d'un ERP

Concernant le déploiement d'un ERP, celui-ci est la plupart du temps **client/serveur** comme le décrit le schéma ci-dessous⁽³⁾ :

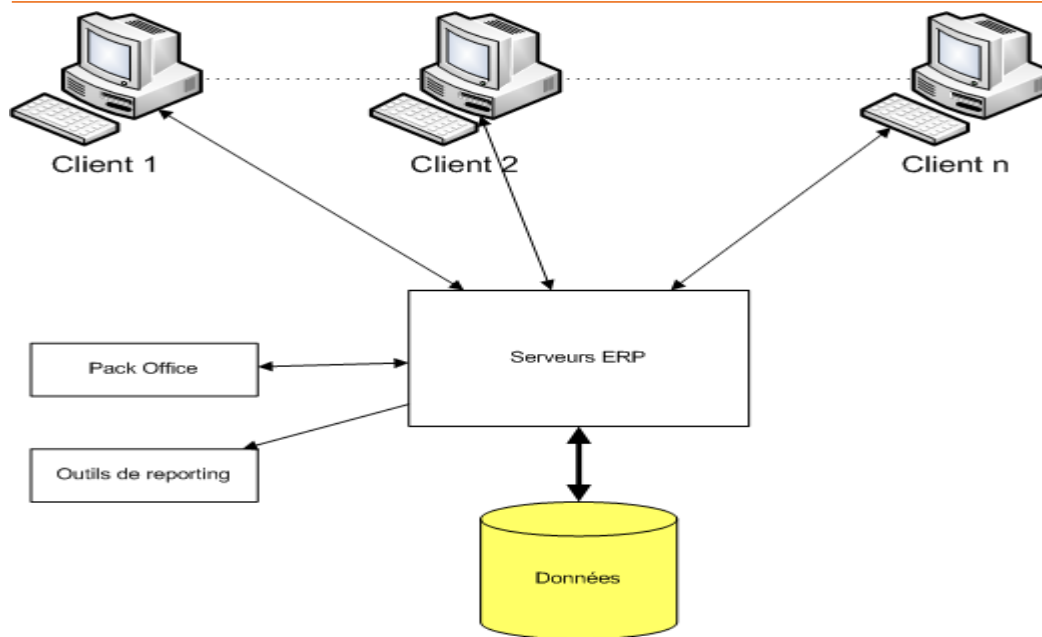


Figure 3: Architecture technique d'un ERP

L'ERP est donc sur serveur. La majorité des ERP sont couplés à une **base de données ORACLE**. De plus, les ERP sont compatibles **pack Office**, en particulier pour Powerpoint et Excel. En effet, le premier étant utile pour personnaliser les bureaux ERP en fonction de l'entreprise et le second pour effectuer les imports/exports de données. Enfin, les ERP sont aussi compatibles avec des **outils de reporting** (CrystalReport en général). Le reporting étant utilisé en particulier pour le module de gestion relation client.

II.9 Les avantages et inconvénients d'un ERP

Les bénéfices d'un ERP pour l'entreprise sont les suivants

- Eviter la redondance d'informations entre différents SI de l'entreprise ;
- Disposer d'un outil multilingue et multidevises (très adapté aux multinationales) ;
- Eviter des restitutions d'informations divergentes entre différents services et donc apaiser les conflits qui en résultaient ;
- Une meilleure coordination des services et du coup un meilleur suivi du processus de commande qui inclut la prise de commande, l'enregistrement d'une sortie de stock, l'expédition de la commande et l'émission d'une facture ;
- Une meilleure maîtrise des stocks ;
- Une normalisation de la gestion des Ressources Humaines, en particulier pour les entreprises qui gèrent de nombreuses entités, parfois géographiquement dispersées.

Quant aux inconvénients, même s'ils sont moins nombreux que les avantages, ils sont néanmoins à prendre en compte.

- L'investissement peut être important en fonction de la taille de l'entreprise et de ses besoins ;
- La mise en œuvre peut être complexe et longue si le projet n'est pas bien piloté et la conduite du changement accompagnée ;
- Certains modules de l'ERP peuvent être moins performants qu'un développement spécifique interne ou un logiciel spécialisé ;

- Besoin d'adapter parfois certains processus de l'organisation ou de l'entreprise au logiciel.

Conclusion

Ce chapitre clôture la première partie de notre travail et avait pour objectif de faire une étude générale des concepts liés à notre sujet. La partie suivante portera sur l'analyse et la conception de notre projet.

DEUXIEME PARTIE : ANALYSE ET CONCEPTION

Cette partie a pour objectif de présenter l'analyse et la conception de notre projet. Nous y présenterons les méthodes de conception et les choix techniques pour l'aboutissement de notre travail.

Chapitre III : CHOIX DE LA METHODE D'ANALYSE ET DE CONCEPTION

Introduction

Une méthode d'analyse et de conception adaptée est nécessairement pour aller des besoins à l'implémentation d'un logiciel. Nous allons présenter les différentes méthodes d'analyse et conception, puis choisir celle qui convient le mieux pour la réalisation de notre progiciel de gestion intégré.

III.1 Les méthodes d'analyse et de conception

Une méthode d'informatisation est composée :

- de modèles : ce sont des spécimens de résultats intermédiaires pouvant être produit par une étape du processus ;
- de langages : pour élaborer les spécifications, et faciliter leur communication ;
- d'une démarche : processus pour effectuer les travaux préconisés, étape par étape.

Méthodes cartésiennes (fonctionnelles)

Avec ces méthodes, le système étudié est vu comme un ensemble de fonctions. Le processus de conception est vu comme un développement linéaire. Il y a décomposition systématique des fonctions principales en sous fonctions, elles-mêmes décomposées en sous-fonctions jusqu'à un niveau considéré élémentaire. En insistant exclusivement sur les fonctions ces méthodes finissent par occulter la prise en compte des données dans le système. SADT (Structured-Analysis-Design-Technique) est un exemple des méthodes cartésiennes.

Méthodes systémiques

L'un des objectifs des méthodes systémiques est de corriger les failles des méthodes cartésiennes en incorporant les données. Elles définissent différents niveaux de préoccupation ou d'abstraction et proposent de nombreux modèles complémentaires. Les méthodes systémiques sont souvent spécialisées pour la conception d'un certain type de systèmes tels que les systèmes de conception de bases de données relationnelles (MERISE, AXIAL..). Toutefois, ces méthodes juxtaposent les données et les traitements sans véritablement les intégrer.

Méthodes orientée-objet

Les méthodes objets viennent intégrer véritablement les données aux fonctions dans le concept objet. Pour un objet donné, les données et les fonctions sont traitées ensemble. Les méthodes objets consistent à créer une représentation informatique des éléments du monde réel auxquels on s'intéresse, sans se préoccuper de l'implémentation, ce qui signifie indépendamment d'un langage de programmation. Il s'agit donc de déterminer les objets présents et d'isoler leurs données et les fonctions qui les utilisent. Pour cela des méthodes ont été mises au point. Entre 1970 et 1990, de nombreux analystes ont mis au point des approches orientées objets, si bien qu'en 1994 il existait plus de 50 méthodes objets. Toutefois seules trois(3) méthodes ont véritablement émergé :

- la méthode OMT de Rumbaugh ;

- la méthode BOOCH'93 de Booch ;
- la méthode OOSE de Jacobson.

Les promoteurs de ces trois (3) méthodes se sont fixés pour objectif de créer une méthode qui fasse la synthèse des éléments indispensables pour faire du développement objet. Sur le plan du langage, ils ont créé le langage UML pour la modélisation orientée objet. Pour les processus, le consensus se fait sur les processus unifiés qui sont des méta-processus à partir desquels le chef du projet doit décrire son véritable processus, en prenant en compte les spécificités du problème posé, les outils et les personnes impliquées. Ce processus réel construit sur le modèle qu'est le processus unifié, s'exprime en termes d'itérations.

Méthodes agiles

Les méthodes de développement dites « méthodes agiles » (en anglais Agile Modeling) visent à réduire le cycle de vie du logiciel (donc accélérer son développement) en développant une version minimale, puis en intégrant les fonctionnalités par un processus itératif basé sur une écoute cliente et des tests tout au long du cycle de développement.

L'origine des méthodes agiles est liée à l'instabilité de l'environnement technologique et au fait que le client est souvent dans l'incapacité de définir ses besoins de manière exhaustive dès le début du projet. Le terme « agile » fait ainsi référence à la capacité d'adaptation aux changements de contexte et aux modifications de spécifications intervenant pendant le processus de développement. En 2001, 17 personnes mirent ainsi au point le manifeste agile dont la traduction est la suivante :

- individus et interactions plutôt que processus et outils ;
- développement logiciel plutôt que documentation exhaustive ;
- collaboration avec le client plutôt que négociation contractuelle ;
- ouverture au changement plutôt que suivi d'un plan rigide.

Grâce aux méthodes agiles, le client est un pilote à part entière de son projet et obtient très vite une première mise en production de son logiciel. Ainsi, il est possible d'associer les utilisateurs dès le début du projet.

III.2 Choix d'une méthode

Compte tenu de la complexité de notre projet, nous avons adopté SCRUM comme canevas pour l'application des procédés et techniques de développement. Toutefois, pour la réalisation des sprints (qui sera défini plus loin), nous allons utiliser un processus UP. En effet, SCRUM donne la liberté à l'équipe de développement de choisir comment mener l'activité d'un sprint ; l'essentiel est de transformer les items du backlog choisis, en incrément de produit potentiellement livrable.

III.3 Pourquoi adopter les méthodes agiles ?

Les méthodes Agiles consistent en un ensemble de pratiques imaginées pour pallier les difficultés rencontrées dans les cycles de développement en cascade ou en V, encore omniprésentes.

Les méthodes traditionnelles prônent un enchaînement séquentiel des différentes activités, depuis les spécifications jusqu'à la validation du système, selon un planning préétabli. Elles visent à mieux prédire la façon dont les choses « devraient » se passer. Malheureusement, cette vision rassurante est bien loin de la réalité des projets. Les activités d'ingénierie ne sauraient se succéder strictement sans qu'aucun changement ne vienne perturber un planning qui n'a de durée de vie que le temps de le prononcer.

La conséquence est que plus de 80% des projets exécutés selon ces méthodologies connaissent des retards, des dépassements budgétaires, quand ils ne finissent pas en échec total, pour n'avoir pas su satisfaire les attentes des clients.

Ces problèmes sont liés à plusieurs caractéristiques fondamentales de ces anciennes méthodologies :

- Le rôle joué par le client : le client intervient principalement au moment du lancement du projet, à quelques jalons majeurs parfois espacés de plusieurs mois, et surtout en fin de projet pour la réception et la recette du système. Cet « effet tunnel » conduit à une solution souvent inadaptée et de piètre qualité.
- Le mode contractuel forfaitaire qui :
 - durcit les relations entre client et fournisseur,
 - rend le passage de témoin long et douloureux à la fin du projet.
- Une trop grande standardisation : des activités d'ingénierie, dont l'enchaînement se révèle souvent inefficace. Formellement, les contrôles d'avancement et de qualité ne peuvent être menés que sur la base de documents dans les premières étapes, et bien des organisations sont devenues des usines à produire de la documentation au lieu de produire de la valeur (fonctions logicielles) pour les clients et les utilisateurs.
- Le passage de relais : entre les phases successives dans lesquelles œuvrent des équipes différentes, généralise une relation de type client-fournisseur et n'encourage ni l'empathie ni l'esprit d'équipe, bien au contraire. Chaque transition se traduit par une perte de temps, de savoir, d'informations ou de responsabilité.

A contrario, les méthodes Agiles préconisent :

- L'adoption d'un cycle itératif et incrémental : permettant à une équipe de s'adapter au contexte ainsi qu'aux changements qui ne manquent pas de survenir au cours d'un projet.
- L'implication du client : dans le développement, permettant au client et à l'utilisateur de donner leur feedback quant au devenir de l'application en cours de développement, annulant ainsi tout « effet tunnel ».
- La définition d'objectifs à court terme : qui permet de maintenir une pression constante mais supportable sur l'équipe, alors qu'au début d'un cycle en V chacun a l'impression d'avoir suffisamment de temps devant lui et subit finalement une pression énorme à l'approche de la livraison.
- La collaboration entre les personnes et l'intégration des équipes : qui combat les fameux passages de relais en rassemblant dans un même espace toutes les énergies et la compétence de personnes centrées sur l'application à réaliser. Plus aucune barrière et des tâches définies par l'équipe au meilleur moment, c'est-à-dire quand on en a besoin, plutôt qu'au début du projet.

- La livraison d'un produit opérationnel : de bonne qualité parce que souvent testé, doté de la seule documentation strictement nécessaire, et répondant à coup sûr aux vrais besoins des utilisateurs puisqu'il est régulièrement soumis à leur feedback.

III.4 Présentation de Scrum⁽⁴⁾

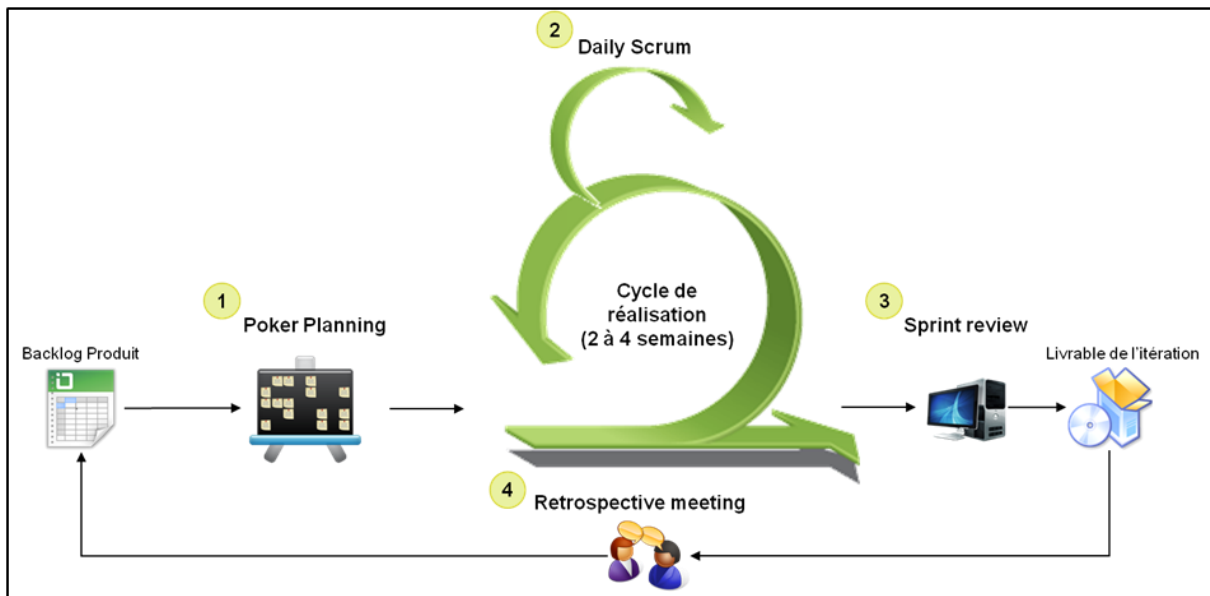


Figure 4: Présentation Scrum

III.4.1 Définition de Scrum

Scrum est un cadre de travail permettant de répondre à des problèmes complexes et changeants, tout en livrant de manière productive et créative des produits de la plus grande valeur possible.

Scrum est utilisé depuis le début des années 1990 pour gérer le développement de produits complexes. Scrum n'est pas en soi un processus ni une méthode de développement de produits ; c'est un canevas pour l'application de divers procédés et techniques de développement. Scrum met en évidence l'efficacité relative des pratiques de gestion et de développement de produit en place, de sorte que ces dernières puissent être améliorées.

Scrum se compose de plusieurs éléments que sont l'Équipe Scrum et ses rôles associés, les événements, les artefacts et les règles. Chaque élément a une raison d'être spécifique qui le rend indispensable à la réussite de l'application de Scrum. Les règles de Scrum sont les modalités qui lient événements, rôles et artefacts entre eux.

III.4.2 Théorie de Scrum

Scrum se base sur la théorie du contrôle empirique de processus, ou l'empirisme. L'empirisme soutient que les connaissances proviennent de l'expérience et d'une prise de décision basée sur des faits connus. Scrum utilise une approche itérative et incrémentale pour optimiser la prédictibilité et pour contrôler le risque.

Trois piliers soutiennent l'implémentation d'un contrôle empirique de processus : la transparence, l'inspection et l'adaptation.

A. Transparence

Les aspects importants du processus doivent être visibles à ceux qui sont responsables des retombées. La transparence requiert la définition d'un standard commun pour ces aspects afin que les observateurs partagent une compréhension commune de ce qui est observé.

À titre d'exemple :

- Un langage commun décrivant le processus doit être partagé par tous les participants ;
- Ceux qui effectuent le travail et ceux qui en acceptent le résultat doivent partager une définition commune de « Terminé ».

B. Inspection

Les utilisateurs de Scrum doivent fréquemment inspecter les artefacts Scrum et l'état d'avancement par rapport à un objectif de Sprint (*Sprint Goal*) afin de détecter les écarts indésirables. La fréquence de ces inspections ne devrait pas gêner le travail en cours. Ces inspections sont bénéfiques lorsqu'elles sont effectuées de manière diligente sur les lieux du travail par les personnes qualifiées.

C. Adaptation

Si un inspecteur détermine qu'un ou plusieurs aspects du processus dérivent hors des limites acceptables, et que le produit qui en résulte sera inacceptable, le processus ou le matériel utilisé par le processus doit être ajusté. Un ajustement doit être fait dès que possible afin de minimiser le risque d'autres dérives.

Scrum prescrit quatre occasions formelles d'inspection et d'adaptation, tel que décrit dans la section Événements Scrum de ce document :

- Planification de Sprint (Sprint Planning)
- Mêlée quotidienne (Daily Scrum)
- Revue de Sprint (Sprint Review)
- Rétrospective de Sprint (Sprint Retrospective)

III.4.3 L'Équipe Scrum

L'Équipe Scrum comprend un propriétaire de produit (*Product Owner*), une Équipe de Développement (*Development Team*) et un Scrum Master. Les Équipes Scrum (*Scrum Teams*) sont auto-organisées et pluridisciplinaires. Les équipes auto-organisées choisissent la meilleure façon d'accomplir leur travail, au lieu d'être dirigées par des personnes externes à l'équipe. Les équipes pluridisciplinaires ont toutes les compétences nécessaires pour effectuer le travail sans dépendre de personnes n'appartenant pas à l'équipe. Scrum définit un modèle d'équipe optimisant la flexibilité, la créativité et la productivité.

Les Équipes Scrum livrent des produits de manière itérative et incrémentale, maximisant ainsi les occasions de rétroaction. Les livraisons incrémentales d'un produit « Terminé » assurent la disponibilité d'une version fonctionnelle et potentiellement utile du produit.

A. Le Product Owner

Le Product Owner est responsable de maximiser la valeur du produit et du travail de l'Équipe de Développement. La façon de jouer ce rôle peut varier grandement selon les entreprises, les Équipes Scrum et les individus.

Le Product Owner est la seule personne responsable de gérer le carnet de produit (*Product Backlog*). La gestion du Product Backlog comprend :

- Exprimer clairement les items du Product Backlog ;
- Ordonner les items du Product Backlog pour mieux réaliser les objectifs et missions ;
- Optimiser la valeur du travail effectué par l'Équipe de Développement ;
- S'assurer que le Product Backlog est visible, transparent, et clair pour tous, et qu'il montre ce sur quoi l'Équipe de Développement travaillera prochainement ;
- S'assurer que l'Équipe de Développement comprend adéquatement les items du Product Backlog.

B. L'Équipe de Développement

L'Équipe de Développement est constituée de professionnels qui livrent à chaque Sprint un incrément potentiellement livrable du produit. Seuls les membres de l'Équipe de Développement créent l'incrément.

Les équipes de développement sont structurées et habilitées par l'entreprise à organiser et gérer leur propre travail. La synergie résultante optimise l'efficacité et l'efficacité globale des équipes de développement.

L'Équipe de Développement possède les caractéristiques suivantes :

- Elle est auto-organisée. Nul (même pas le Scrum Master) n'indique à l'Équipe de Développement comment transformer les items du Product Backlog en incréments de fonctionnalités potentiellement livrables.
- Elle est pluridisciplinaire, avec toutes les compétences nécessaires pour créer un incrément du produit ;
- Scrum ne reconnaît aucun titre aux membres de l'Équipe de Développement autre que celui de développeur, indépendamment du travail effectué par cette personne ; il n'y a pas d'exception à cette règle ;
- Scrum ne reconnaît pas d'équipes à l'intérieur de l'Équipe de Développement indépendamment des domaines spécifiques qui doivent être couverts tels que l'exécution de tests ou l'analyse fonctionnelle ; il n'y a pas d'exception à cette règle ; et,
- Les membres de l'Équipe de Développement peuvent détenir individuellement des compétences et des centres d'intérêt spécifiques, mais c'est l'Équipe de Développement dans son ensemble qui est tenue responsable.

C. Le Scrum Master

Le Scrum Master est responsable de s'assurer que Scrum est compris et mis en œuvre. Les Scrum Masters remplissent leur rôle en s'assurant que l'Équipe Scrum adhère à la théorie, aux pratiques et aux règles de Scrum.

Le Scrum Master est un leader au service de l'Équipe Scrum. Le Scrum Master aide ceux qui sont externes à l'Équipe Scrum à comprendre lesquelles de leurs interactions avec l'Équipe Scrum sont bénéfiques et lesquelles ne le sont pas. Le Scrum Master aide tout le monde à changer ces interactions pour maximiser la valeur créée par l'Équipe Scrum.

III.4.4 Les événements Scrum

Les événements prescrits par Scrum créent de la régularité et minimisent la nécessité d'autres réunions non prévues. Tous les événements sont limités dans le temps, de telle sorte que chaque événement ait une durée maximale. Une fois le Sprint commencé, sa durée est fixe et ne peut être écourtée ou prolongée. Les autres événements peuvent se terminer une fois

les objectifs de ceux-ci atteints, dans la mesure où suffisamment de temps a été alloué sans que du gaspillage ait lieu dans le processus.

Mis à part le Sprint, qui contient tous les autres événements, chaque événement de Scrum est une occasion d'inspecter et d'adapter quelque chose. Ces événements sont conçus spécifiquement pour permettre transparence et inspection. Le défaut d'inclure n'importe lequel de ces événements résulte en une réduction de la transparence et constitue une occasion ratée d'inspection et d'adaptation.

A. Le Sprint

Au cœur de Scrum, le Sprint a une durée d'un mois ou moins au cours duquel une version « terminée », utilisable et potentiellement livrable du logiciel est créée. Il est préférable que les Sprints gardent une durée constante tout au long de l'initiative de développement. Un nouveau Sprint débute immédiatement après la conclusion du précédent.

Les Sprints contiennent et sont constitués de la planification du Sprint (*Sprint Planning*), des mêlées quotidiennes (*Daily Scrums*), des activités de développement, de la revue du Sprint (*Sprint Review*) et de la rétrospective du Sprint (*Sprint Retrospective*).

Pendant le sprint :

- L'objectif du sprint est fixe ; les changements qui le remettent en cause ne sont donc pas permis ;
- Les objectifs de qualité sont maintenus ; ils ne sont jamais revus à la baisse ;
- Le périmètre peut être clarifié et renégocié entre le Product Owner et l'Équipe de Développement selon ce que l'Équipe Scrum apprend.

Chaque Sprint peut être considéré comme un projet qui dure au maximum un mois. À l'instar du projet, le Sprint est utilisé pour réaliser un objectif. La définition des fonctionnalités à développer, la conception et le plan flexible qui guidera le développement, l'activité de développement en soi et le produit résultant sont associés au Sprint.

La durée d'un Sprint est limitée à un mois calendaire. Lorsque l'échéance d'un Sprint est trop éloignée, la définition de ce qui est à développer peut changer et cela peut accroître la complexité et le risque. Les Sprints amènent de la prévisibilité en forçant une inspection et adaptation du progrès vers l'atteinte d'un objectif au moins mensuellement. Les Sprints limitent également le risque financier à un mois calendaire.

B. Mêlée quotidienne

La mêlée quotidienne (*Daily Scrum*) est un événement limité à 15 minutes au cours duquel l'équipe de développement synchronise ses activités et crée un plan pour les prochaines 24 heures. Pour ce faire, l'équipe inspecte le travail effectué depuis la dernière mêlée quotidienne et envisage le travail qui peut être réalisé d'ici à la prochaine.

La mêlée a lieu tous les jours à la même heure et au même endroit afin de réduire la complexité. Durant la réunion, les membres de l'Équipe de Développement décrivent :

- Ce qu'ils ont réalisé hier qui a aidé l'Équipe de Développement à atteindre l'objectif du Sprint ;
- Ce qu'ils réaliseront aujourd'hui pour aider l'Équipe de Développement à atteindre l'objectif du Sprint ;
- Les obstacles qui, selon eux, les empêchent ou empêche l'Équipe de Développement d'atteindre l'objectif du Sprint.

L'Équipe de Développement utilise la mêlée quotidienne pour inspecter sa progression vers l'objectif du Sprint et l'achèvement du travail prévu au Sprint Backlog

C. Revue du Sprint

Une revue de Sprint est tenue à la fin du Sprint pour inspecter l'incrément réalisé et adapter le Product Backlog si nécessaire. Pendant la réunion de revue de Sprint, l'Équipe Scrum et les parties prenantes échangent sur ce qui a été fait durant le Sprint. En se basant là-dessus, et en considérant les changements au Product Backlog effectués durant le Sprint, les participants collaborent pour déterminer les prochains items ayant le plus de valeur qui pourraient être faits. Cette réunion se veut informelle, pas une réunion de pilotage, et la présentation de l'incrément est destinée à susciter des réactions et à favoriser la collaboration.

Cette réunion est limitée à quatre heures pour un Sprint d'un mois. Pour les Sprints moins long, la revue de Sprint dure généralement moins longtemps.

D. Rétrospective de Sprint

La rétrospective de Sprint (*Sprint Retrospective*) est une occasion pour l'Équipe Scrum de s'inspecter et de créer un plan d'amélioration qui sera mis en place au cours du Sprint suivant. La rétrospective de Sprint survient après la revue de Sprint et avant la prochaine réunion de planification de Sprint. C'est une réunion limitée à trois heures pour les Sprints d'un mois. Pour les Sprints moins longs, la réunion dure habituellement moins longtemps.

III.4.5 Les artefacts de Scrum

Les artefacts de Scrum représentent soit du travail soit de la valeur fournissant ainsi de la transparence et des opportunités pour l'inspection et l'adaptation. Les artefacts de Scrum sont spécialement conçus pour maximiser la transparence d'informations essentielles afin que tous en aient la même compréhension.

A. Product Backlog

Le Product Backlog est une liste ordonnée de tout ce qui pourrait être requis dans le produit et est l'unique source des besoins pour tous les changements à effectuer sur le produit. Le Product Owner est responsable du Product Backlog dans son contenu, sa disponibilité et son ordonnancement.

Un Product Backlog n'est jamais complet. Ses toutes premières moutures ne font qu'esquisser les besoins tels qu'initialement connus et compris. Le Product Backlog évolue au fur et à mesure que le produit et le contexte dans lequel il sera utilisé évoluent. Le Product Backlog est dynamique ; il change constamment pour identifier ce que le produit requiert pour être approprié, compétitif et utile. Tant et aussi longtemps qu'un produit existe, son Product Backlog correspondant existe.

Le Product Backlog liste toutes les fonctionnalités, besoins, améliorations et correctifs correspondant aux changements devant être appliqués au produit lors de livraisons futures. Les items du Product Backlog incluent une description, un ordre, une estimation de l'effort et de la valeur.

B. Sprint Backlog

Le Sprint Backlog est l'ensemble des items sélectionnés pour le Sprint plus un plan pour livrer l'incrément du produit et réaliser l'objectif du Sprint. Le Sprint Backlog est une prévision que l'Équipe de Développement fait de la fonctionnalité qui sera présente dans le

prochain incrément et le travail nécessaire pour livrer cette fonctionnalité dans un incrément « terminé ».

C. Incrément

L'incrément est constitué des éléments du Product Backlog terminés pendant le sprint ainsi que de la valeur cumulative des incréments livrés dans les sprints précédents. A la fin d'un Sprint, le nouvel incrément doit être « terminé », ce qui implique qu'il doit être dans un état utilisable et qu'il correspond à la définition de « terminé » de l'Équipe de Développement. Il doit être dans un état utilisable, sans égard à la décision du Product Owner de le rendre disponible ou non.

III.4.6 Risques de la méthodologie

On peut citer trois risques, mais il en existe bien d'autres :

- Flaccid scrum : Le terme « flaccid scrum » est utilisé par Martin Fowler en 2009 pour identifier une pratique erronée de scrum dans laquelle la qualité logicielle est négligée et le produit développé accumule de la dette technique. Fowler indique que même si scrum ne décrit pas quelles pratiques techniques mettre en place, la méthode n'encourage pas l'absence de telles pratiques et que les « scrummers » de premier plan ont toujours insisté sur l'utilisation de pratiques techniques efficaces. La négligence de l'aspect technique de l'activité de développement et l'augmentation de la dette technique qui en résulte sont des signes d'une mauvaise utilisation de scrum.
- Un scrum master directif : Il n'est pas rare que le scrum master soit un chef de projet déguisé ou officiel, et qu'il applique un contrôle trop strict sur les membres de son équipe. Même si l'on demande aux membres de l'équipe de s'impliquer et d'être coresponsables du projet, en réalité ils ne le sont pas. Le scrum master, qui est dans ce cas-là le chef de projet, a le dernier mot pour tout et peut effectuer des choix arbitraires, ce qui peut frustrer les membres de l'équipe. En outre, il n'endosse plus les responsabilités du scrum master : être le protecteur de l'équipe, objectif, garde-fou, garant de la méthodologie appliquée, etc.
Les bénéfices de l'agilité disparaissent, et l'on voit apparaître une défiance de la part de l'équipe vis-à-vis du scrum master, des conflits à l'intérieur même de l'équipe, des non-dits, du désengagement, l'oubli des bonnes pratiques. L'une des conséquences est une baisse de la qualité, élément pourtant non négociable dans un projet agile : « Un logiciel qui fonctionne ».
- La communication constructive laisse la place aux reproches : La surexposition dans scrum prend sa source dans les différentes réunions d'équipe telles que la quotidienne matinale, censée informer et apporter des solutions aux problèmes de la veille. Sur certains projets, ces réunions se transforment en blâme des membres n'ayant pas réussi à mener à bien leurs tâches de la veille. Les conséquences dans ce cas peuvent prendre la forme de discussions houleuses ou de conflits plus ou moins durables entre membres de l'équipe. On constate aussi parfois à l'inverse une manière différente d'appréhender la réunion quotidienne : non-dit ou propos vagues en ce qui concerne les problèmes rencontrés. Ainsi, les réunions peuvent se passer de façon plus courtoise mais un risque de ne plus chercher le support de l'équipe en cas de difficulté pourrait apparaître. Ce qui va à l'encontre même des principes de scrum.

III.5 Présentation du processus unifié (UP)

A. Définition et principes

Le processus unifié est un processus de développement logiciel : il regroupe les activités à mener pour transformer les besoins d'un utilisateur en système logiciel. (Mouadh, 2010). C'est un patron de processus pouvant être adapté à une large classe de systèmes logiciels, à différents domaines d'application, à différents types d'entreprises, à différents niveaux de compétences et à différentes tailles d'entreprises.

Le processus unifié s'appuie sur les principes suivants :

- **Centré sur l'architecture** : L'architecture logicielle représente les aspects statiques et dynamiques du système. L'architecture émerge des besoins de l'entreprise, tels qu'ils sont exprimés par les utilisateurs et reflétés par les cas d'utilisation. L'architecture propose une vue d'ensemble de la conception faisant ressortir les caractéristiques essentielles en laissant de côté les détails secondaires. Il faut noter que tout produit est à la fois forme et fonction. L'une ou l'autre isolément ne saurait suffire. Les cas d'utilisation et l'architecture doivent s'équilibrer pour créer un produit réussi. (LAHLOU, 2010)
- **Piloté par les cas d'utilisation** : Un cas d'utilisation représente une fonctionnalité qui satisfait un besoin d'un utilisateur. Le processus suit une voie spécifique, en procédant par une série d'enchaînement d'activités, dérivées d'un cas d'utilisation. Un cas d'utilisation est analysé, conçu, implémenté et enfin testé. (LAHLOU, 2010)
- **Itératif et incrémental** : Vu que les projets à réaliser sont de plus en plus complexes et grands, l'idée est de découper le travail en mini projets. Chacun d'entre eux représente une itération qui donne lieu à un incrément. Les itérations désignent des étapes de l'enchaînement d'activités, tandis que les incréments correspondent à des stades de développement du produit. (Jackobson, et al., 1999)
- **Orienté par la réduction des risques** : L'analyse des risques doit être présente à tous les stades de développement d'un système. Il est important de bien évaluer les risques des développements afin d'aider à la bonne prise de décision. Du fait de l'application du processus itératif, UP contribue à la diminution des risques au fur et à mesure du déroulement des itérations successives. (Gabay, et al., 2008)

B. Les phases du processus unifié

Le processus unifié se déroule en quatre phases, création, élaboration, construction et transition. Chaque phase répète un nombre de fois une série d'itérations. Et chaque itération est composée de cinq activités : capture des besoins, analyse, conception, implémentation et test. (Jackobson, et al., 1999)

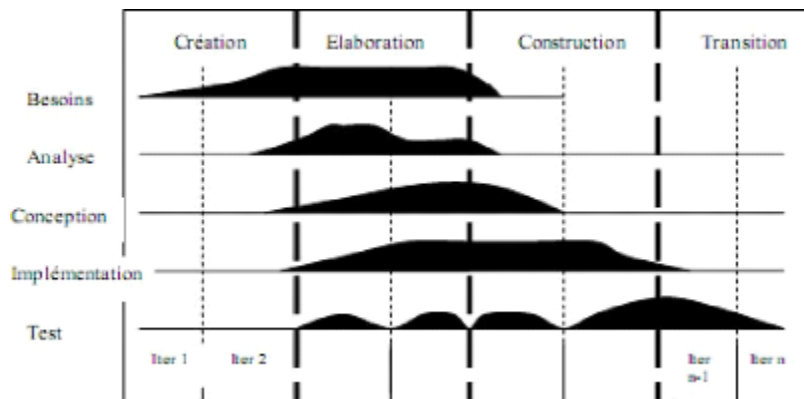


Figure 5: Phase du processus unifié

❖ Création

C'est la première phase du processus unifié. Il s'agit de délimiter la portée du système, c'est-à-dire tracer ce qui doit figurer à l'intérieur du système et ce qui doit rester à l'extérieur, identifier les acteurs, lever les ambiguïtés sur les besoins et les exigences nécessaires dans cette phase. Il s'agit aussi d'établir une architecture candidate, c'est-à-dire que pour une première phase, on doit essayer de construire une architecture capable de fonctionner. Dans cette phase, il faut identifier les risques critiques susceptibles de faire obstacles au bon déroulement du projet. (Jackobson, et al., 1999)

❖ Elaboration

C'est la deuxième phase du processus. Après avoir compris le système, dégagé les fonctionnalités initiales, précisé les risques critiques, le travail à accomplir maintenant consiste à stabiliser l'architecture du système. Il s'agit alors de raffiner le modèle initial de cas d'utilisation, voire capturer de nouveaux besoins, analyser et concevoir la majorité des cas d'utilisation formulés, et si possible implémenter et tester les cas d'utilisation initiaux. (Jackobson, et al., 1999)

❖ Construction

Dans cette phase, il faut essayer de capturer tous les besoins restants car il n'est pratiquement plus possible de le faire dans la prochaine phase. Ensuite, continuer l'analyse, la conception et surtout l'implémentation de tous les cas d'utilisation. A la fin de cette phase, les développeurs doivent fournir une version exécutable du système. (Jackobson, et al., 1999)

❖ Transition

C'est la phase qui finalise le produit. Il s'agit au cours de cette phase de vérifier si le système offre véritablement les services exigés par les utilisateurs, détecter les défaillances, combler les manques dans la documentation du logiciel et adapter le produit à l'environnement (mise en place et installation). (Jackobson, et al., 1999)

III.6 Présentation d'UML (Unified Modified Language)

Né de la fusion des «méthodes objet» dominantes (OMT, Booch et OOSE), puis normalisé par l'Object Management Group (OMG) en 1997, UML est rapidement devenu un standard incontournable. UML n'est pas à l'origine des concepts objet, mais il en donne une définition plus formelle et apporte la dimension méthodologique qui faisait défaut à l'approche objet.

UML est un langage de modélisation au sens de la théorie des langages. Il contient de ce fait les éléments constitutifs de tout langage, à savoir : des concepts, une syntaxe et une sémantique. De plus, UML a choisi une notation supplémentaire : il s'agit d'une forme visuelle fondée sur des diagrammes. Si l'unification d'une notation est secondaire par rapport aux éléments constituant le langage, elle reste cependant primordiale pour la communication et la compréhension.

UML s'articule maintenant autour de 13 diagrammes différents, dont 4 nouveaux diagrammes introduits par UML 2.0. Chacun d'eux est dédié à la représentation d'un système logiciel suivant un point de vue particulier. Par ailleurs, UML modélise le système suivant deux modes de représentation : l'un concerne la structure du système pris « au repos », l'autre concerne sa dynamique de fonctionnement. Les deux représentations sont nécessaires et complémentaires pour schématiser la façon dont est composé le système et comment ses composantes fonctionnent entre elles ⁽⁶⁾.

Les six (6) diagrammes structurels sont :

- Diagramme de classes : il montre les briques de base statiques (classes, associations, interfaces, attributs, opérations, généralisations, etc.) ;
- Diagramme d'objets : il montre les instances des éléments structurels et leurs liens à l'exécution ;
- Diagramme de packages : il montre l'organisation logique du modèle et les relations entre packages ;
- Diagramme de structure composite : il montre l'organisation interne d'un élément statique complexe ;
- Diagramme de composants : il montre des structures complexes, avec leurs interfaces fournies et requises ;
- Diagramme de déploiement : il montre le déploiement physique des « artefacts » sur les ressources matérielles.

Les sept (7) diagrammes comportementaux sont :

- Diagramme de cas d'utilisation : il montre les interactions fonctionnelles entre les acteurs et le système à l'étude ;
- Diagramme de vue d'ensemble des interactions : il fusionne les diagrammes d'activité et de séquence pour combiner des fragments d'interaction avec des décisions et des flots ;
- Diagramme de séquence : il montre la séquence verticale des messages passés entre objets au sein d'une interaction ;

- Diagramme de communication : il montre la communication entre objets dans le plan au sein d'une interaction ;
- Diagramme de temps : il fusionne les diagrammes d'état et de séquence pour montrer l'évolution de l'état d'un objet au cours du temps ;
- Diagramme d'activité : il montre l'enchaînement des actions et décisions au sein d'une activité ;
- Diagramme d'état : il montre les différents états et transitions possibles des objets d'une classe.

Conclusion

Ce chapitre nous a permis de présenter les différentes méthodes de conception de systèmes informatiques. Nous avons pu à travers cette étude faire un choix sur la méthode que nous allons adopter. Il s'agit de coupler le langage UML au processus SCRUM.

Chapitre IV : ANALYSE ET DE CONCEPTION

Introduction

Dans ce chapitre, nous allons effectuer l'analyse et la conception de notre ERP en utilisant SCRUM comme canevas, tout en y associant le langage UML.

IV.1 Etude de l'existant

IV.1.1 Description de l'existant

La version initiale de 2isoftmanaging qui nous été remise permettait de gérer essentiellement la facturation clients. Ainsi, il était possible d'éditer une facture proforma, consulter la liste des factures proforma, transformer une facture proforma en facture définitive, consulter la liste des factures (payées et impayées), imprimer la facture.

IV.1.2 Critique de l'existant

Après une analyse minutieuse de la version initiale de 2iSoftmanaging, nous faisons les remarques suivantes :

- L'architecture utilisée ne facilitait pas la maintenance de l'application. En effet, il n'y a pas de séparation entre les instructions d'accès à la base de données et celles permettant de générer l'interface utilisateurs. En d'autres termes, il n'y a pas de séparation de couches. Or la séparation en couche est un aspect important sinon fondamental pour la mise en place d'un ERP. Par ailleurs, l'API « MySQL » utilisé pour accéder à la base de données est obsolète. Ce qui fait que pour que l'application puisse fonctionner sur un serveur utilisant un SGBD MySQL récent, il faudra réécrire toute les instructions d'accès à la base de données. Et comme il n'y a pas de séparation de couche, c'est toute l'application qu'il va falloir parcourir pour modifier les instructions d'accès à la base de données.
- Le contrôle de gestion (Possibilité de bloquer les utilisateurs non autorisés, Traçabilité des opérations) est un élément incontournable dans la réalisation d'un ERP. Cependant, il est inexistant au niveau de la version initiale de 2iSoftmanaging.
- L'absence d'un bon niveau de paramétrage.
- L'existence de faille XSS au niveau de l'application.
- L'absence de documentation pour la poursuite du projet ou la maintenance de l'application.

Perspective

L'application existante ne constitue qu'un module du travail attendu. Sans oublier que toute l'architecture technique doit être repensée et refaite.

IV.2 Déroulement des sprints

IV.2.1 Sprint 0

Dans ce sprint, nous allons faire le choix de l'architecture technique, identifier les acteurs, définir le diagramme de contexte de l'application, définir le Sprint backlog ; construire le noyau de l'application.

A. Choix de l'architecture technique :

Pour la réalisation de notre application, nous avons choisi l'architecture MVC (Modèle, Vue, Contrôleur) afin de disposer d'une architecture souple, maintenable et facile d'exploitation. Par ailleurs, l'application à développer sera basée sur le modèle client-serveur.

Le Modèle

Le Modèle représente le comportement de l'application, à savoir : traitements des données, interactions avec la base de données, etc. Il décrit les données manipulées par l'application et définit les méthodes d'accès.

Il offre des méthodes pour mettre à jour ces données (insertion, suppression, changement). Il offre aussi des méthodes pour récupérer ses données. Dans le cas de données importantes, le modèle peut autoriser plusieurs vues partielles des données.

La vue

La vue représente l'interface utilisateur. C'est en fait la partie « Interface graphique » avec laquelle l'utilisateur interagit. Sa première tâche est de présenter les résultats renvoyés par le modèle qui lui sont passés par le contrôleur. Sa seconde tâche est de recevoir toutes les actions de l'utilisateur et de les envoyer au contrôleur. La vue n'effectue aucun traitement, elle se contente d'afficher les résultats des traitements effectués par le modèle ;

Le contrôleur

Le contrôleur prend en charge la gestion des événements de synchronisation pour mettre à jour la vue ou le modèle et les synchroniser. Il reçoit tous les événements de la vue et enclenche les actions à effectuer.

Si une action nécessite un changement des données, le contrôleur demande la modification des données au modèle afin que les données affichées se mettent à jour. D'après le patron de conception observateur/observable, la vue est un « observateur » du modèle qui est lui « observable ». Certains événements de l'utilisateur ne concernent pas les données mais la vue. Dans ce cas, le contrôleur demande à la vue de se modifier.

Le contrôleur ne modifie aucune donnée. Il analyse la requête du client et se contente d'appeler le modèle adéquat et de renvoyer la vue correspondant à la demande.

B. Identification des acteurs

Il s'agit ici de bien identifier les acteurs qui auront à interagir avec notre système. Un acteur est un humain, une machine, ou un tout autre système (logiciel) qui ne fait pas partie du système à mettre en place, mais qui participe à son fonctionnement général par une interaction. Identifier les acteurs consiste à identifier les limites de notre projet et les profils des différentes personnes ou machines qui vont interagir avec notre système. Ci-dessous les différents acteurs de notre système et les éventuelles relations qui les relient :

- L'Administrateur : il est chargé de la gestion des utilisateurs et du contrôle du droit d'accès. Il peut également effectuer les actions de la RAF.
- Le Responsable Administratif et Financier (RAF) : il est chargé de toutes les opérations en dehors de la gestion des utilisateurs et du contrôle du droit d'accès.
- L'employé : il est chargé d'annoter les tâches qui lui sont affectées

C. Diagramme de contexte statique

Cet élément de modélisation proposé par le langage UML donne un aperçu graphique du système avec les intervenants directs sur celui-ci :

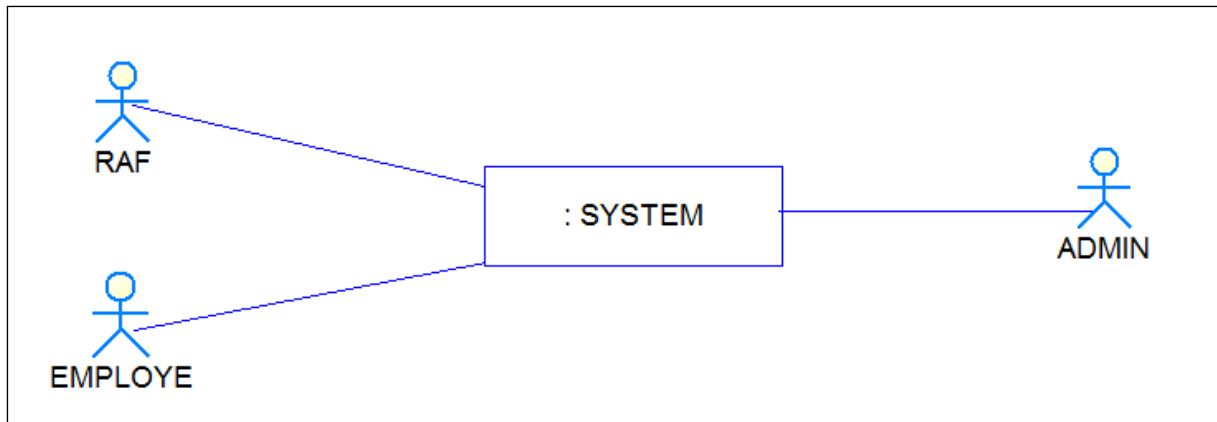


Figure 6: Diagramme de contexte statique

D. Définition du Product backlog

Thème	User story
Gestion des utilisateurs	En tant qu'administrateur, je voudrais : Ajouter / consulter / modifier / supprimer un utilisateur
	En tant qu'administrateur, je voudrais : Attribuer / retirer des privilèges à un utilisateur
	En tant qu'administrateur, je voudrais : Consulter les opérations effectuées par un utilisateur
Gestion des fournisseurs	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un fournisseur
Gestion des clients	En tant qu'administrateur, je voudrais : Ajouter / consulter / modifier / supprimer un client
Gestion des produits	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un produit
	En tant qu'administrateur, je voudrais : Augmenter / consulter / modifier le stock d'un produit
Gestion de la trésorerie	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un compte
	En tant que RAF, je voudrais : Consulter l'historique des opérations d'un compte sur une période donnée
	En tant que RAF, je voudrais : Transférer des fonds d'un compte à un autre
	En tant que RAF, je voudrais : Consulter l'historique des transferts d'un compte à un autre sur une période donnée
gestion des établissements financiers	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un établissement
Gestion des taxes et	En tant qu'administrateur, je voudrais :

cotisations	Ajouter / consulter / modifier / supprimer types de taxes et cotisation
	En tant qu'administrateur, je voudrais : Calculer le montant d'une taxe donnée payée aux fournisseurs à sur période donnée
	En tant que RAF, je voudrais : Calculer le montant d'une taxe donnée reçu des clients sur une période donnée
	En tant que RAF, je voudrais : Calculer le montant d'une taxe donnée dû à un établissement sur une période donnée
	En tant que RAF, je voudrais : Calculer le montant des cotisations dû à un établissement financier sur une période donnée
	En tant que RAF, je voudrais : Consulter l'historique des paiements de taxe et cotisation
Gestion des Achats	En tant que RAF, je voudrais : Etablir / consulter / modifier / annuler une commande fournisseur
	En tant que RAF, je voudrais : Enregistrer / consulter / modifier / annuler une livraison
	En tant que RAF, je voudrais : Enregistrer / consulter / modifier / annuler les paiements de factures
Gestion des Ventes	En tant que RAF, je voudrais : Etablir / consulter / modifier / annuler un devis
	En tant que RAF, je voudrais : Etablir / consulter / modifier / annuler une commande client
	En tant que RAF, je voudrais : Etablir une commande à partir d'un devis
	En tant que RAF, je voudrais : Enregistrer / consulter / modifier / annuler les paiements de factures
	En tant que RAF, je voudrais : Générer et imprimer une facture
	En tant que RAF, je voudrais : Générer et envoyer une facture par mail
Gestion des tâches	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer / Clore une tâche
	En tant que RAF, je voudrais : Imputer/Affecter une tâche
	En tant qu'Employé, je voudrais : Affecter/ Annoter une tâche
	En tant que RAF, je voudrais : Consulter les tâches hors délais

Tableau 1:Product backlog

IV.2.2 : Construction du cœur de notre système

A1.Fonctionnement général du système : Architecture MVC

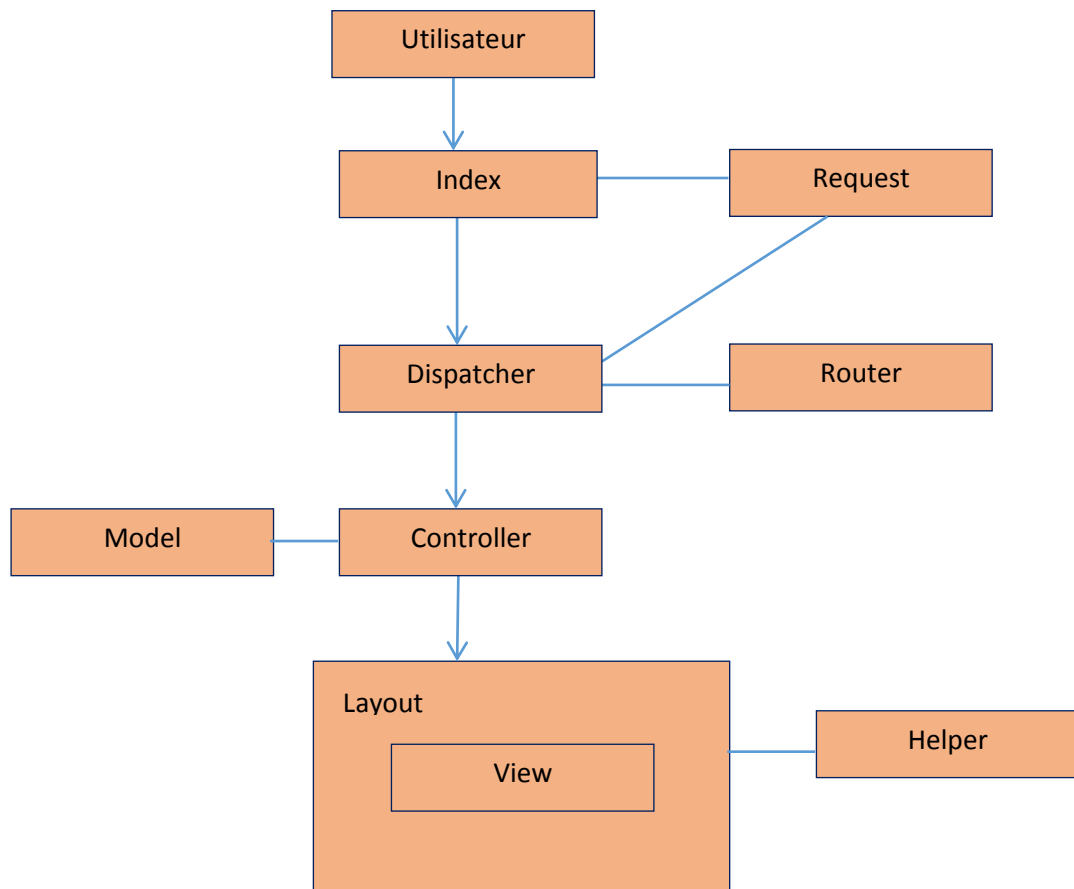


Figure 7: Fonctionnement du système

Toutes les URL saisies seront redirigées vers la page index (/WebRoot/index.php). Index va inclure le Dispatcher.

- Le Dispatcher est un objet qui va permettre la redirection et initialiser les bons composants en fonction de l'URL saisie.
- Le Dispatcher va récupérer un objet Request ; Request est un objet qui va contenir toutes les infos concernant la requête de l'utilisateur.
- Le Dispatcher va récupérer l'URL mais lui ne sait pas l'interpréter ; il va faire appel au Router pour décomposer l'URL.
- A partir du Router, le Dispatcher saura quel composant appelé et quelle action effectuée. Il initialise le Controller et appelle l'action.
- Le Controller fera éventuellement appel au Model et envoie ses variables à la Vue.
- La Vue affiche la page à l'utilisateur

A2. Structure du dossier de l'application

Il est composé de quatre (4) dossiers et d'un fichier .htaccess ;

- Config (dossier)
- Core (dossier)
- Modules (dossier)
- WebRoot (dossier)
- .htaccess (fichier)

Le dossier Config

Il contient un seul fichier : Conf.php dans lequel on a défini la classe abstraite Conf ; cette classe possède un seul attribut : un tableau dont chaque élément est une configuration pour se connecter à la base de donnée ; sachant que nous avons une configuration par profil d'utilisateur, c'est-à-dire que nous avons une configuration pour se connecter en tant qu'administrateur du système, une configuration pour la connexion en tant que simple utilisateur, etc.

Le dossier Core

C'est le noyau du système. Il contient 6 fichiers qui sont :

- Request.php
- Router.php
- Dispatcher.php
- Model.php
- Controller.php
- Includes.php

a) Request.php

Il contient la définition de la classe Request. Cette classe a un seul attribut : l'URL saisie par l'utilisateur

b) Router.php

Il contient la définition de la classe Router, qui est une classe abstraite. On l'utilise juste pour parser les URL.

Toutes nos URL sont de la forme :

Adresse du serveur/dossier de l'application/Controller/action/ langue

- Controller par défaut = Controller de connexion
- Action par défaut = Authentification
- Langue par défaut = français

c) Dispatcher.php

Il contient la définition de la classe Dispatcher. Le Dispatcher est le véritable chef d'orchestre ;

- Il instancie le Request et envoie ce dernier au Router pour qu'il soit parsé.
- Après cette opération, il vérifie si le Controller à initialiser (d'après le Request) est valide (existe). Si oui, il l'instancie grâce à la méthode LoadController ; sinon il affiche un message d'erreur grâce à la méthode Error ; Toutefois, la méthode LoadController vérifie d'abord si l'utilisateur est authentifié ; dans le cas échéant, il est redirigé vers la page Connexion (en chargeant le Controller connexion).
- Si l'utilisateur est bien authentifié et que le Controller demandé est valide, Le Dispatcher passe à la vérification de l'action demandée. Il vérifie d'abord si l'action demandée existe au niveau du Controller initialisé ; si oui, il vérifie si l'utilisateur est autorisé à effectuer cette action. En cas de non autorisation, on lui affiche un message de refus d'accès. Si tout se passe bien, l'utilisateur aura accès à la ressource demandée.

d) Model.php

Il contient la définition de la classe Model ; celle-ci assure l'interaction du système avec la base de données. On y trouve la méthode de connexion à la base de donnée (nous avons utilisé PDO), et les méthodes de manipulation de données (select, insert, delete, update). L'avantage de cette classe, est qu'on pourrait migrer vers un autre SGBD sans difficulté.

e) Controller.php

Il contient la définition de la classe Controller. Le Controller contient la définition des actions.

- Il permet l'affichage de la vue et fournit un lexique à celle-ci en fonction de la langue choisie
- Il permet de valider les formulaires au niveau de la vue et sert donc d'interface entre le Model et la vue

NB :

- ✓ La langue par défaut de l'application est définie au niveau du Controller, à travers l'attribut \$DefaultLanguage ; mais aussi au niveau du Router à travers l'attribut \$langue. N'hésitez donc pas à modifier la langue par défaut en cas de besoin.
- ✓ Le Template par défaut est défini au niveau du Controller à travers l'attribut \$theme.
- ✓ Pour ajouter une nouvelle langue à l'application, il faut se référer au Controller principal (qui est dans le dossier Core) ainsi qu'aux Controllers de tous les modules pour mettre à jour le lexique envoyé aux vues ainsi que les messages d'erreur. (Ils sont faciles à repérer, ce sont des tableaux associatifs dont les index sont : fr, en, etc.)

f) Includes.php

Le fichier includes.php est utilisé pour regrouper toutes les inclusions de fichiers. On y trouve les inclusions des fichiers suivants :

- Conf.php
- Request.php
- Router.php
- Controller.php
- Model.php
- Dispatcher.php

Le dossier Modules

Il contient l'ensemble des modules installés (un dossier par module). Chaque module est composé de :

- Un Controller qui a le même nom que le Module suivi de Controller (exemple : ConnexionController)
- Un Model qui a le même nom que le Module suivi de Model
- Un ensemble de vues
- Les dossiers classiques (imgs, css, js) qui sont dans le dossier WebRoot

Le dossier WebRoot

Il contient :

- Le fichier index de notre application
- Le dossier Layout ou sont sauvegarder les templates
- Le dossier Framework ou sont sauvegarder les Framework utilisés (Jquery, Bootstrap, tinymce, etc.)
- Le dossier libs ou sont sauvegarder les librairies (form.php, img.php, captcha.php, etc)
- Les dossiers classiques (imgs, js, css)

Le fichier htaccess

Il contient les règles de réécriture de l'URL.

IV.2.3 Sprint 1 : gestion des utilisateurs

Thème	User story
Gestion des utilisateurs	En tant qu'administrateur, je voudrais : Ajouter / consulter / modifier / supprimer un utilisateur
	En tant qu'administrateur, je voudrais : Attribuer / retirer des privilèges à un utilisateur

Tableau 2: Sprint backlog, gestion des utilisateurs

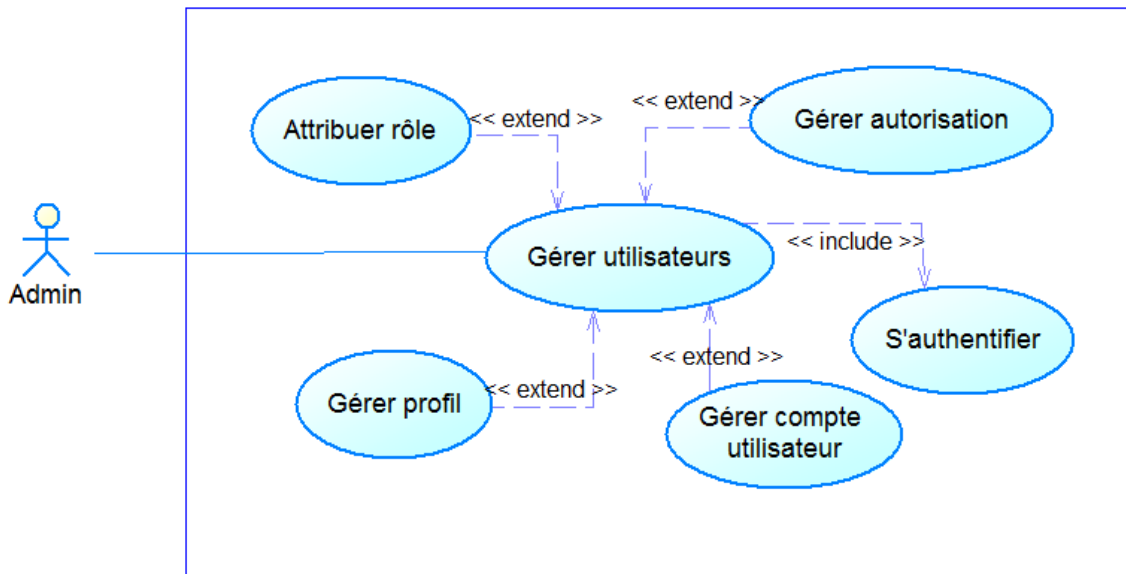


Figure 8: Cas d'utilisation gestion des utilisateurs

Cas d'utilisation « s'authentifier »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : S'authentifier

Résumé : Ce cas d'utilisation permet à un utilisateur de s'authentifier

Date : 25/09/2015

Version : 0.1

Description des enchainements

Scénario nominal

Le système demande à l'utilisateur de saisir son login et son mot de passe

1. L'utilisateur saisit son login et son mot de passe et valide (A1)
2. Le système enregistre les informations saisies et affiche les fonctionnalités accessibles à l'utilisateur

Scénario alternatif

A1.Login ou mot de passe incorrect

1. Le système affiche un message à l'utilisateur l'informant de la situation
2. Retour au point 1 du scénario nominal

Post condition

L'utilisateur est authentifié

L'événement est enregistré dans le journal système

Cas d'utilisation « Créer un compte utilisateur »

Sommaire d'identification

Acteur(s) : Administrateur

Titre : Créer un compte utilisateur

Résumé : Ce cas d'utilisation permet de créer un compte d'utilisateur

Date : 25/09/2015

Version : 0.1

Pré condition

Administrateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'administrateur de saisir les informations relatives au compte utilisateur
2. L'administrateur saisit le nom d'utilisateur, le mot de passe et valide (A1)
3. Le système enregistre les informations fournies et informe l'administrateur

Scénario alternatif

A1. Le nom d'utilisateur existe déjà

1. Le système affiche un message à l'administrateur l'informant de la situation
2. Retour au point 1 du scénario nominal

Post condition

Compte d'utilisateur créé

L'événement est enregistré dans le journal système

Cas d'utilisation « Créer un rôle»

Sommaire d'identification

Acteur(s) : Administrateur

Titre : Créer un rôle

Résumé : Ce cas d'utilisation permet de créer un rôle

Date : 25/09/2015

Version : 0.1

Pré condition

Administrateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'administrateur de saisir le libellé du rôle
2. L'administrateur saisit le libellé du rôle (A1)
3. Le système enregistre l'information fournie et informe l'administrateur

Scénario alternatif

A1. Le rôle existe déjà

1. Le système affiche un message à l'administrateur l'informant de la situation
2. Retour au point 1 du scénario nominal

Post condition

Rôle créé

L'événement est enregistré dans le journal système

Cas d'utilisation « Attribuer rôle »

Sommaire d'identification

Acteur(s) : Administrateur

Titre : Attribuer rôle

Résumé : Ce cas d'utilisation permet d'attribuer un rôle à un utilisateur

Date : 25/09/2015

Version : 0.1

Pré condition

Administrateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche à l'administrateur la liste des utilisateurs et les rôles existants au sein du système
2. L'administrateur sélectionne le(s) rôle(s) à attribuer ainsi que le(s) utilisateur(s) bénéficiaire(s) (A1)
3. Le système enregistre les informations fournies et informe l'administrateur

Scénario alternatif

A1.Rôle déjà attribué

1. Le système affiche un message à l'administrateur l'informant de la situation
2. Retour au point 1 du scénario nominal

Post condition

Le rôle est attribué à l'utilisateur

L'événement est enregistré dans le journal système

Cas d'utilisation « Gérer autorisation »

Sommaire d'identification

Acteur(s) : Administrateur

Titre : Attribuer rôle

Résumé : Ce cas d'utilisation permet de définir les fonctionnalités auxquelles un rôle peut accéder

Date : 25/09/2015

Version : 0.1

Pré condition

Administrateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche à l'administrateur la liste des fonctionnalités regroupées par module et les rôles existants au sein du système
2. L'administrateur sélectionne les fonctionnalités à attribuer au(x) rôle(s) et valide
3. Le système enregistre les informations fournies et informe l'administrateur

Post condition

Les autorisations sont attribuées au rôle est attribué à l'utilisateur

L'événement est enregistré dans le journal système

Diagramme de classe gestion des utilisateurs

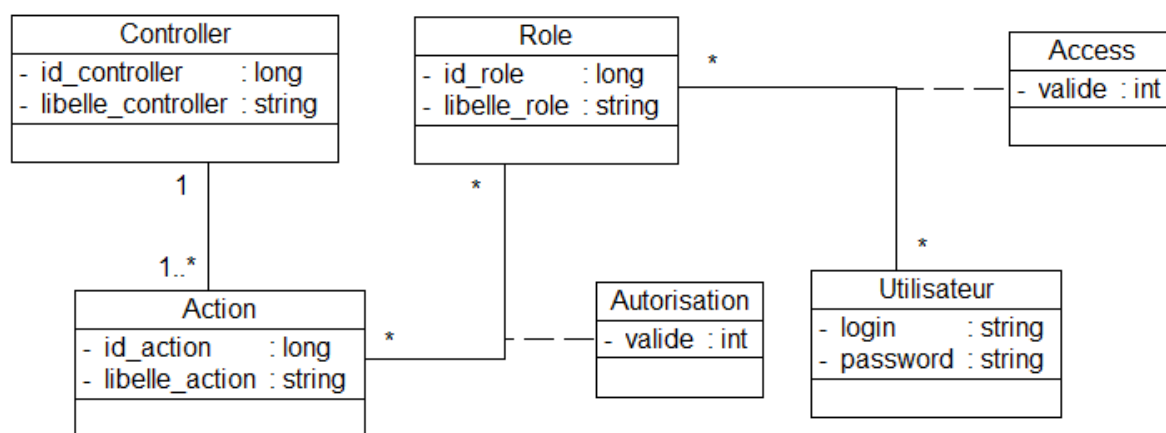


Figure 9: Diagramme de classe, gestion des utilisateurs

IV.2.4 Sprint 2 : Eléments de base

Thème	User story
Gestion des fournisseurs	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un fournisseur
Gestion des clients	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un client
Gestion des produits	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un produit
Gestion de la trésorerie	En tant que RAF, je voudrais : Augmenter / consulter / modifier le stock d'un produit
	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un compte
	En tant que RAF, je voudrais : Consulter l'historique des opérations d'un compte sur une période donnée
	En tant que RAF, je voudrais : Transférer des fonds d'un compte à un autre
gestion des établissements financiers	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer un établissement
Gestion des taxes et cotisations	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer types de taxes et cotisation
	En tant que RAF, je voudrais : Calculer le montant d'une taxe donnée payée aux fournisseurs à sur période donnée
	En tant que RAF, je voudrais : Calculer le montant d'une taxe donnée reçu des clients sur une période donnée

Gestion des taxes et cotisations	
	En tant que RAF, je voudrais : Calculer le montant d'une taxe donnée dû à un établissement sur une période donnée
	En tant que RAF, je voudrais : Calculer le montant des cotisations dû à un établissement financier sur une période donnée
	En tant que RAF, je voudrais : Consulter l'historique des paiements de taxe et cotisation

Tableau 3: Sprint backlog, Eléments de base

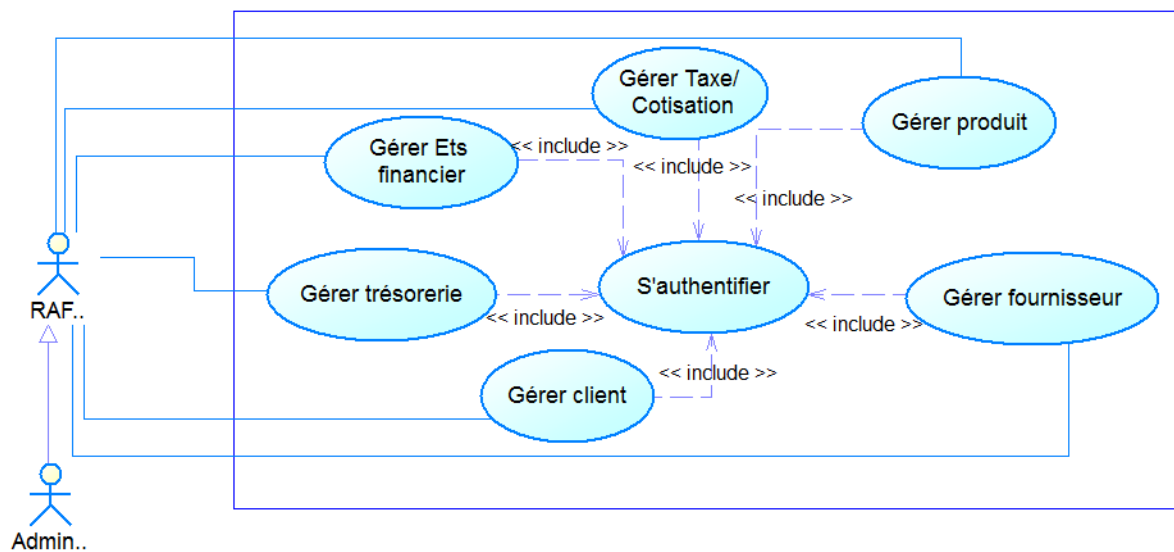


Figure 10: Diagramme des cas d'utilisation, Eléments de base

Cas d'utilisation « Enregistrer client »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer client

Résumé : Ce cas d'utilisation permet d'enregistrer un client

Date : 20/10/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de saisir les informations du client
2. L'utilisateur saisit les informations relatives au client et valide (A1)
3. Le système enregistre les informations et informe l'utilisateur

Scénario alternatif

A1. Les informations fournies sont invalides

1. Le système informe l'utilisateur que les informations saisies sont invalides
2. Retour au point 1 du scénario nominal

Post condition

Client créé

Diagramme de séquence « Enregistrer client »

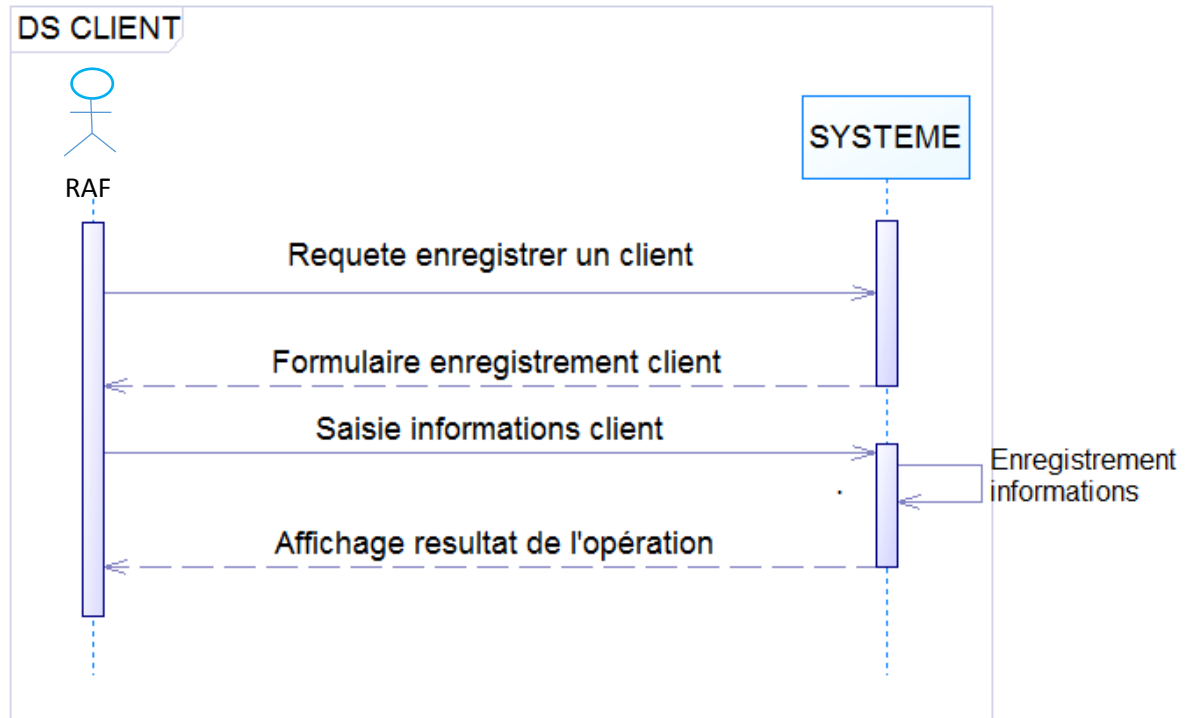


Figure 11: Diagramme de séquence, enregistrer client

Cas d'utilisation « Enregistrer fournisseur »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer fournisseur

Résumé : Ce cas d'utilisation permet d'enregistrer un fournisseur

Date : 20/10/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de saisir les informations du fournisseur
2. L'utilisateur saisit les informations relatives au fournisseur et valide (A1)
3. Le système enregistre les informations et informe l'utilisateur

Scénario alternatif

- A1. Les informations fournies sont invalides
 1. Le système informe l'utilisateur que les informations saisies sont invalides
 2. Retour au point 1 du scénario nominal

Post condition

Fournisseur créé

Diagramme de séquence « Enregistrer fournisseur »

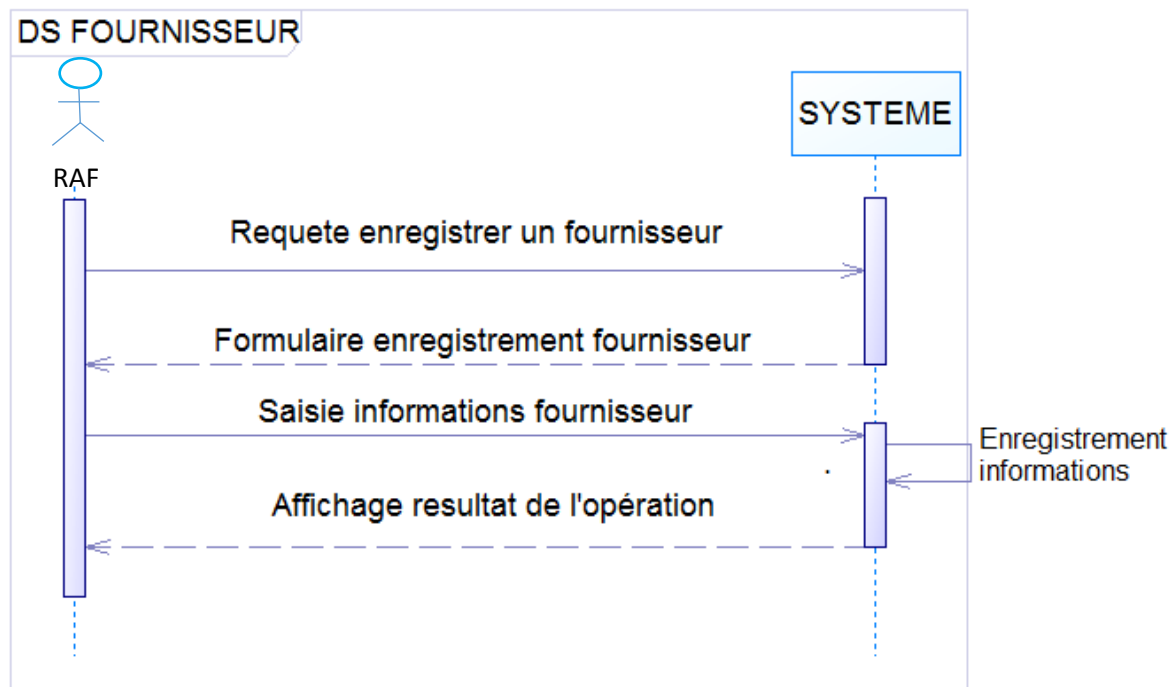


Figure 12: Diagramme de séquence enregistrer fournisseur

Cas d'utilisation « Enregistrer produit »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer fournisseur

Résumé : Ce cas d'utilisation permet d'enregistrer un produit

Date : 20/10/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de saisir les informations du produit
2. L'utilisateur saisit les informations relatives au produit et valide (A1)
3. Le système enregistre les informations et informe l'utilisateur

Scénario alternatif

A1. Les informations fournies sont invalides

1. Le système informe l'utilisateur que les informations saisies sont invalides
2. Retour au point 1 du scénario nominal

Post condition

Produit créé

Diagramme de séquence « Enregistrer produit »

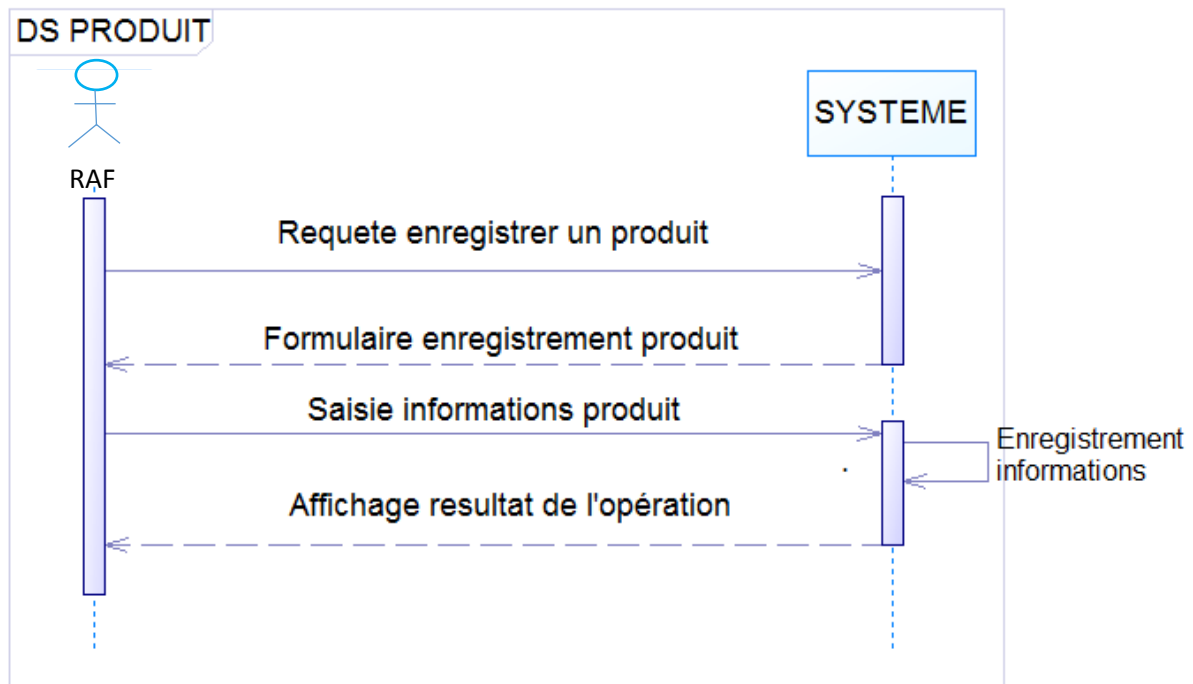


Figure 13: Diagramme de séquence, enregistrer produit

IV.2.5 Sprint 3 : Gestion des Achats

Thème	User story
Gestion des Achats	En tant que RAF, je voudrais : Etablir / consulter / modifier / annuler une commande fournisseur
	En tant que RAF, je voudrais : Enregistrer / consulter / modifier / annuler une livraison
	En que RAF, je voudrais : Enregistrer / consulter / modifier / annuler les paiements de factures

Tableau 4: Sprint backlog, gestion des achats

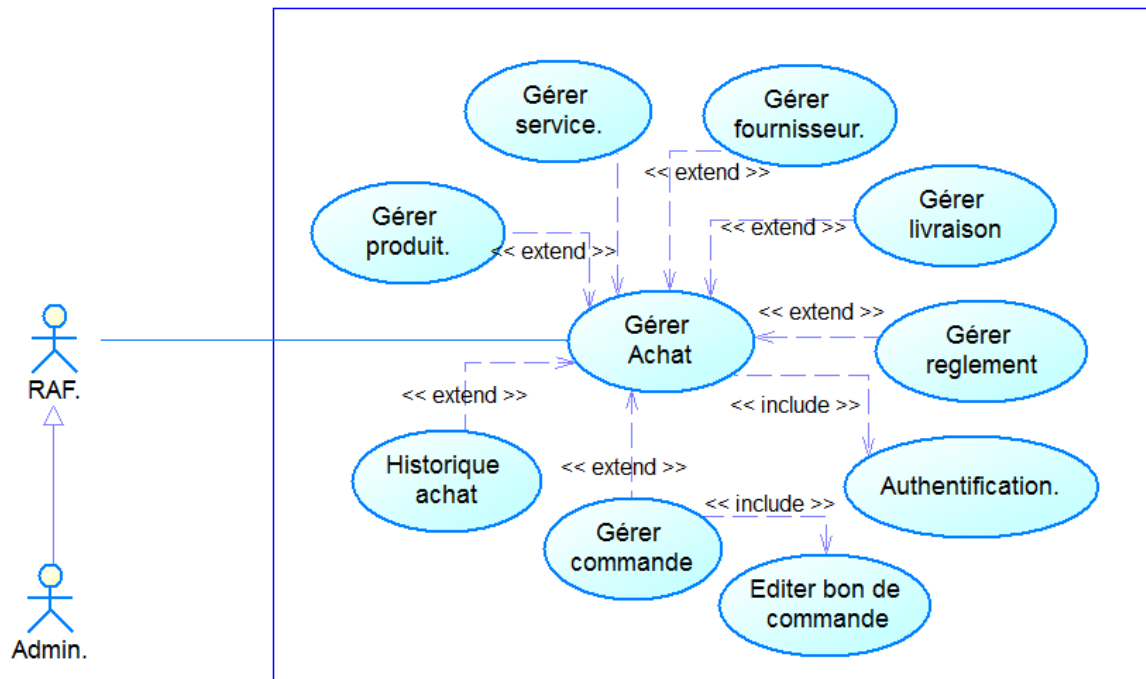


Figure 14: Diagramme des cas d'utilisation, gestion des achats

Cas d'utilisation « Enregistrer commande fournisseur »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer Commande fournisseur

Résumé : Ce cas d'utilisation permet d'enregistrer une commande passée chez un fournisseur

Date : 18/11/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de fournir les informations (produits et services commandés, quantité commandée, ...) concernant la commande
2. L'utilisateur fournit les informations et valide (A1, A2)
3. Le système enregistre la commande et informe l'utilisateur
4. L'utilisateur imprime le bon de commande

Scénario alternatif

A1.Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

A2.Le montant total de la facture est trop élevé

1. Supprimer une ligne de commande ou modifier la quantité d'un produit commandé
2. Retour au point 2 du scénario nominal

Post condition

Commande enregistrée

Diagramme de séquence « Enregistrer commande fournisseur »

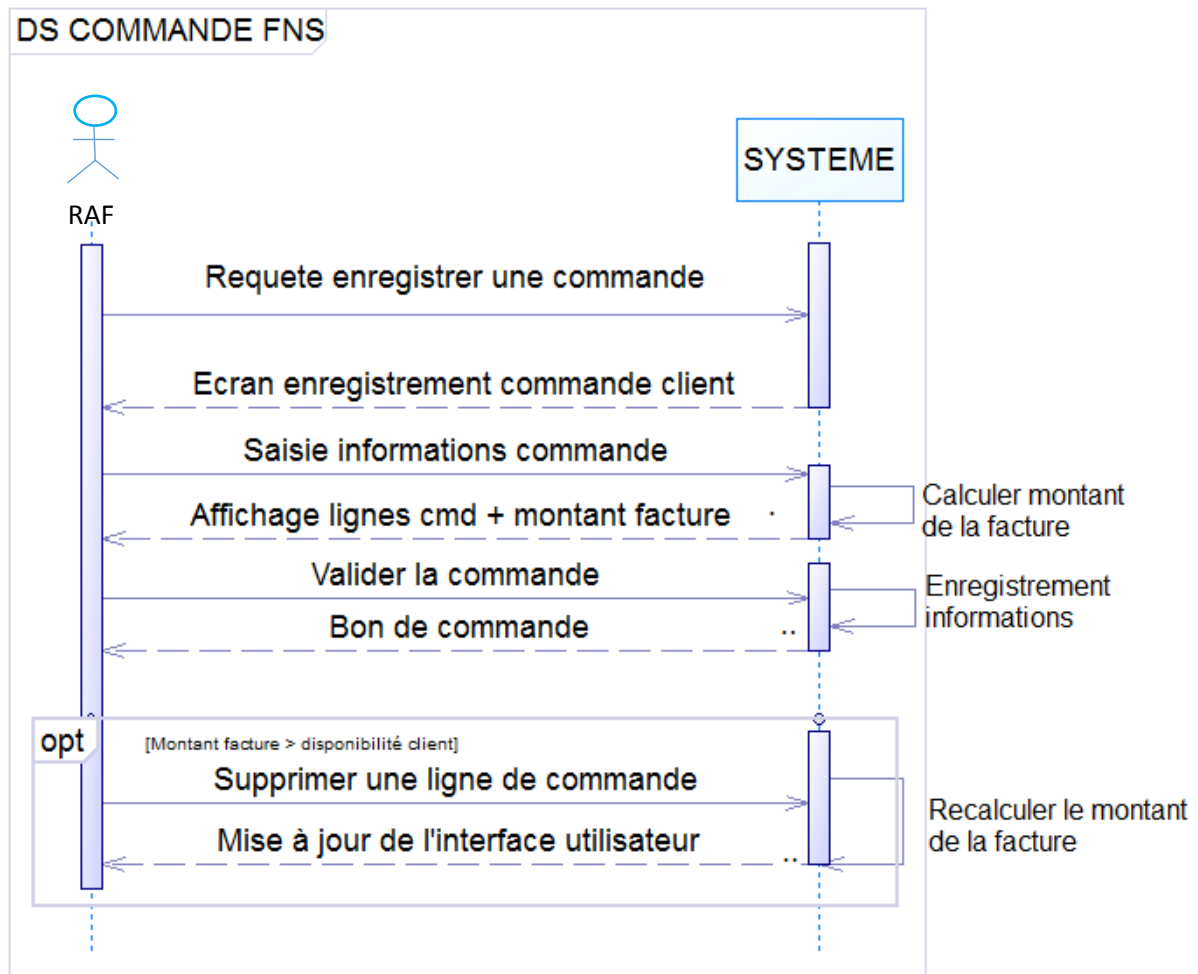


Figure 15: Diagramme de séquence, Enregistrer commande fournisseur

Cas d'utilisation « Enregistrer livraison fournisseur »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer livraison fournisseur

Résumé : Ce cas d'utilisation permet d'enregistrer la livraison d'un fournisseur

Date : 25/11/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des commandes fournisseurs non livrées
2. L'utilisateur sélectionne la commande et saisit les quantités livrées pour les différents produits et services commandés (A1)
3. Le système enregistre la livraison et informe l'utilisateur

Scénario alternatif

A1. Erreur dans la saisie des informations (quantité livrée > quantité commandée)

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

Post condition

Livraison enregistrée

Cas d'utilisation « Enregistrer règlement fournisseur »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer Livraison client

Résumé : Ce cas d'utilisation permet d'enregistrer le règlement d'une facture chez un fournisseur

Date : 28/11/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des commandes fournisseur non réglées
2. L'utilisateur sélectionne la commande
3. L'utilisateur sélectionne le mode de paiement et le compte à débiter
4. L'utilisateur saisit le montant du règlement et valide (A1, A2)
5. Le système enregistre le règlement et informe l'utilisateur

Scénario alternatif

A1. Montant en compte < Montant versement

1. Le système informe l'utilisateur de la situation
2. Retour au point 3 du scénario nominal

A2. Montant en compte > Montant versement et Montant versement > Reste à payer

1. Le système informe l'utilisateur de la situation
2. Retour au point 3 du scénario nominal

Post condition

Règlement enregistré

Diagramme de séquence « Enregistrer règlement fournisseur »

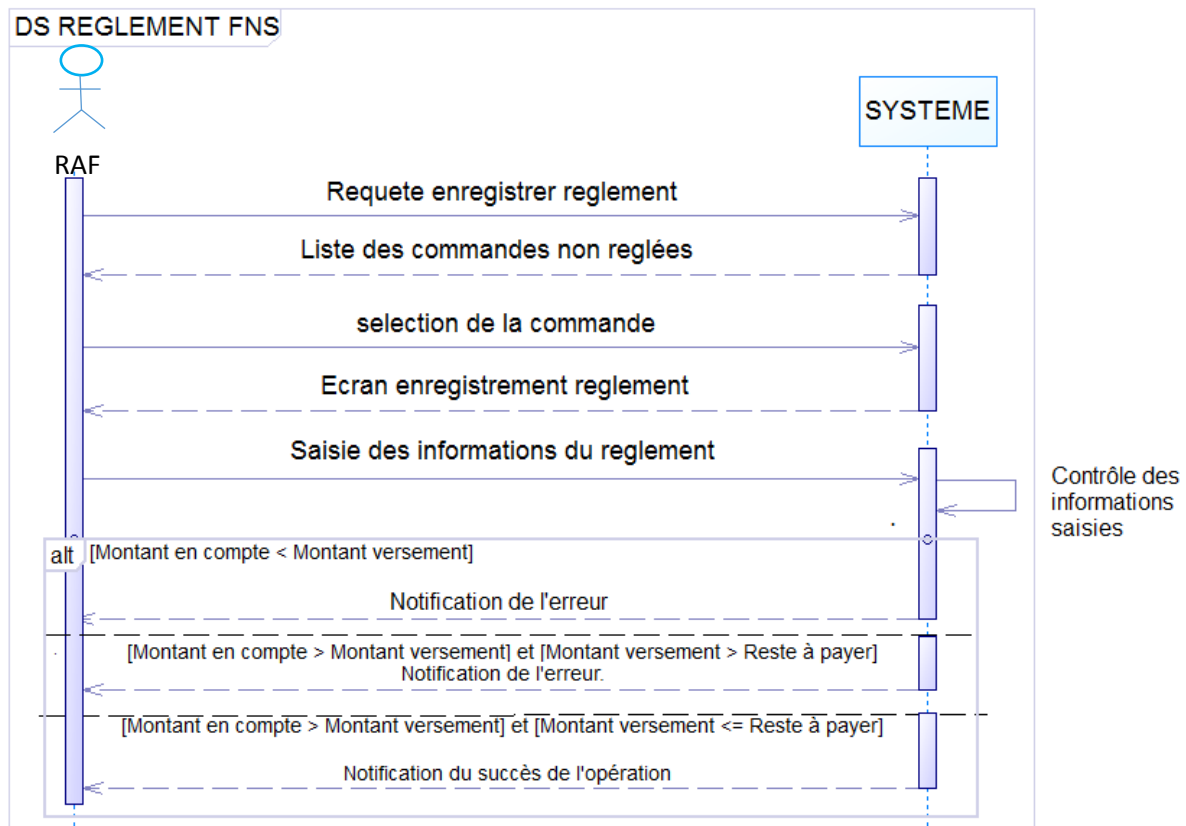


Figure 16: Diagramme de séquence, règlement fournisseur

Diagramme de classe Achat

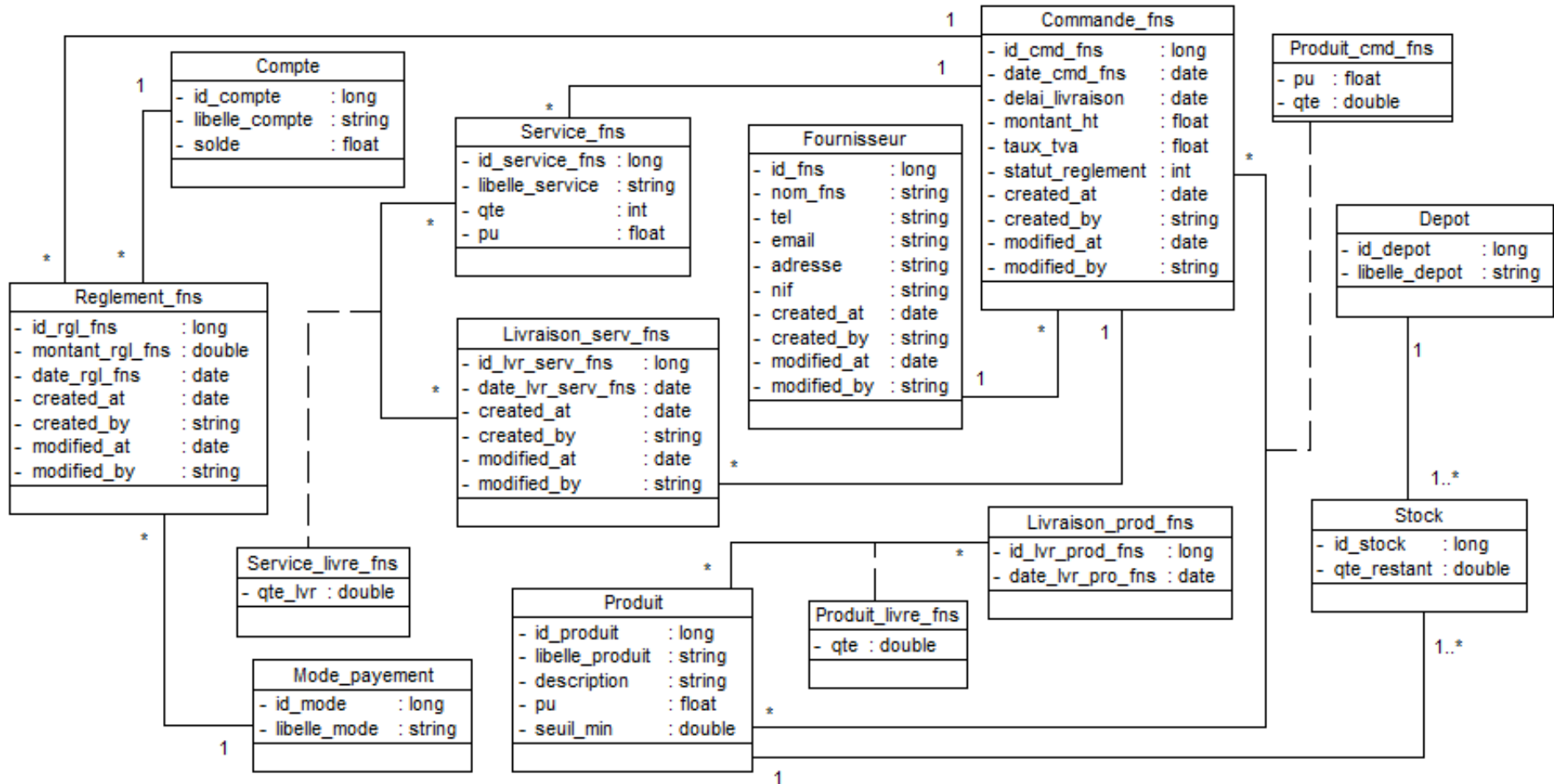


Figure 17: Diagramme de classe, gestion des achats

IV.2.6 Sprint 4 : Gestion des ventes

Thème	User story
Gestion des Ventes	En tant que RAF, je voudrais : Etablir / consulter / modifier / annuler un devis
	En tant que RAF, je voudrais : Etablir / consulter / modifier / annuler une commande client
	En tant que RAF, je voudrais : Etablir une commande à partir d'un devis
	En tant que RAF, je voudrais : Enregistrer / consulter / modifier / annuler les paiements de factures

Tableau 5: Sprint backlog, gestion des ventes

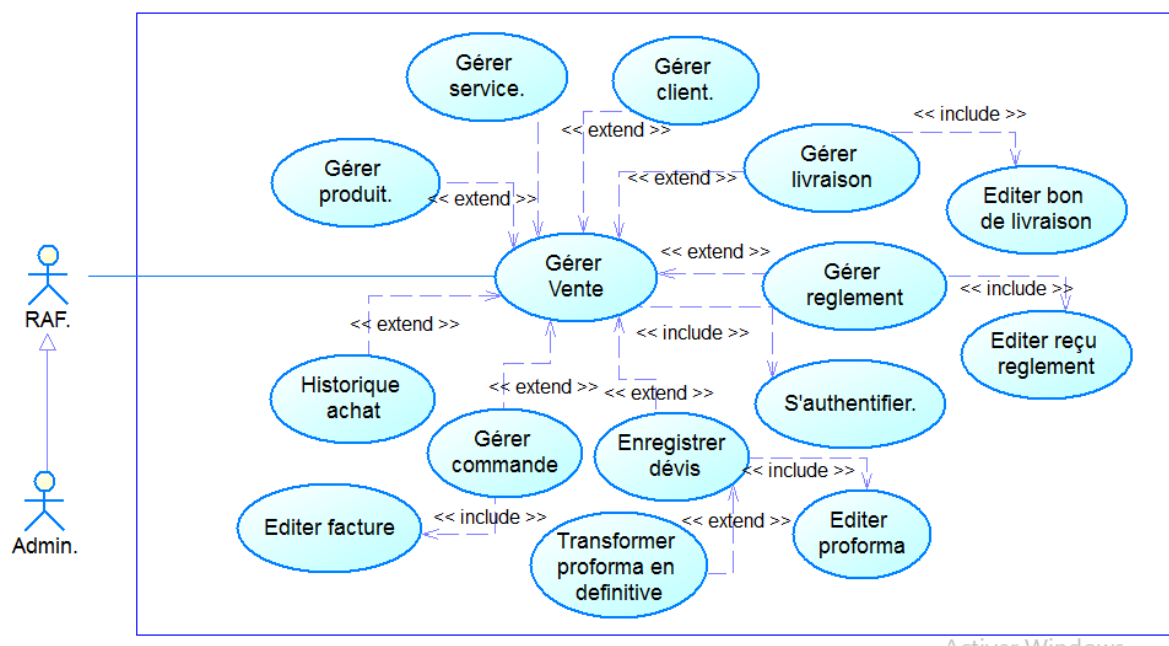


Figure 18: Diagramme des cas d'utilisation, gestion des ventes

NB : Nous avons fait abstraction de l'authentification pour la clarté du modèle

Cas d'utilisation « Enregistrer commande client »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer Commande client

Résumé : Ce cas d'utilisation permet d'enregistrer la commande d'un client

Date : 28/11/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de fournir les informations (produits et services commandés, quantité commandée, ...) concernant la commande du client

2. L'utilisateur fournit les informations et valide (A1, A2)
3. Le système enregistre la commande et informe l'utilisateur
4. L'utilisateur imprime la facture

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

A2. Le montant total de la facture est trop élevé

1. Supprimer une ligne de commande ou modifier la quantité d'un produit commandé
2. Retour au point 2 du scénario nominal

Post condition

Commande enregistrée

Diagramme de séquence « Enregistrer commande client »

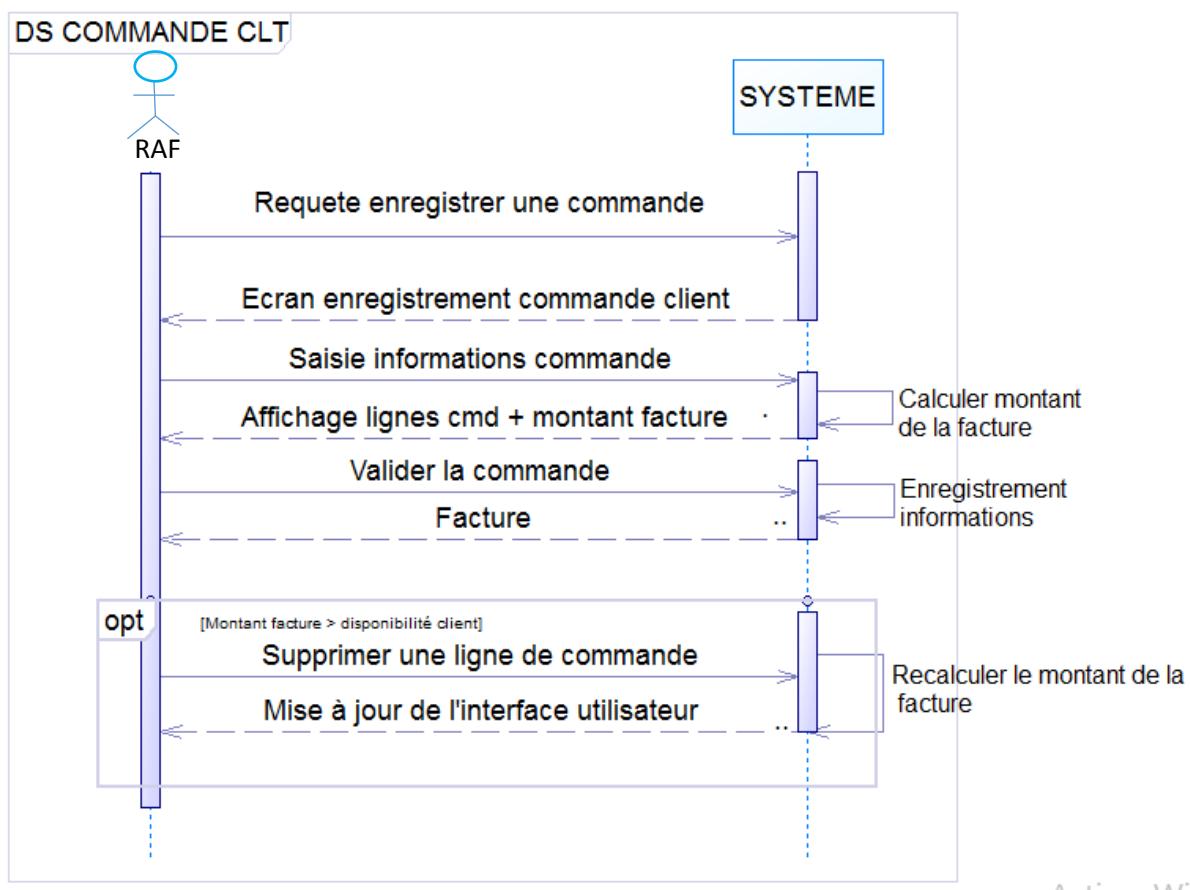


Figure 19: Diagramme de séquence, enregistrer commande client

Cas d'utilisation « Enregistrer un devis »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer un devis

Résumé : Ce cas d'utilisation permet d'enregistrer un devis

Date : 29/11/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de fournir les informations (produits et services demandés, quantité demandée, ...) concernant le devis
2. L'utilisateur fournit les informations et valide (A1, A2)
3. Le système enregistre la demande et informe l'utilisateur
4. L'utilisateur imprime la facture pro forma

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

A2. Le montant total de la facture est trop élevé

1. Supprimer une ligne de produit ou service, ou modifier la quantité d'un produit demandé
2. Retour au point 2 du scénario nominal

Post condition

Devis enregistré

Cas d'utilisation « Enregistrer livraison client »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer Livraison client

Résumé : Ce cas d'utilisation permet d'enregistrer la livraison de produit et service à un client

Date : 02/12/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des commandes clients non livrées
2. L'utilisateur sélectionne la commande et saisit les quantités livrées pour les différents produits et services commandés (A1)
3. Le système enregistre la livraison et informe l'utilisateur
4. L'utilisateur imprime le bon de livraison

Scénario alternatif

A1. Erreur dans la saisie des informations (quantité livrée > quantité commandée)

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

Post condition

Livraison enregistrée

Diagramme de séquence « Enregistrer livraison client »

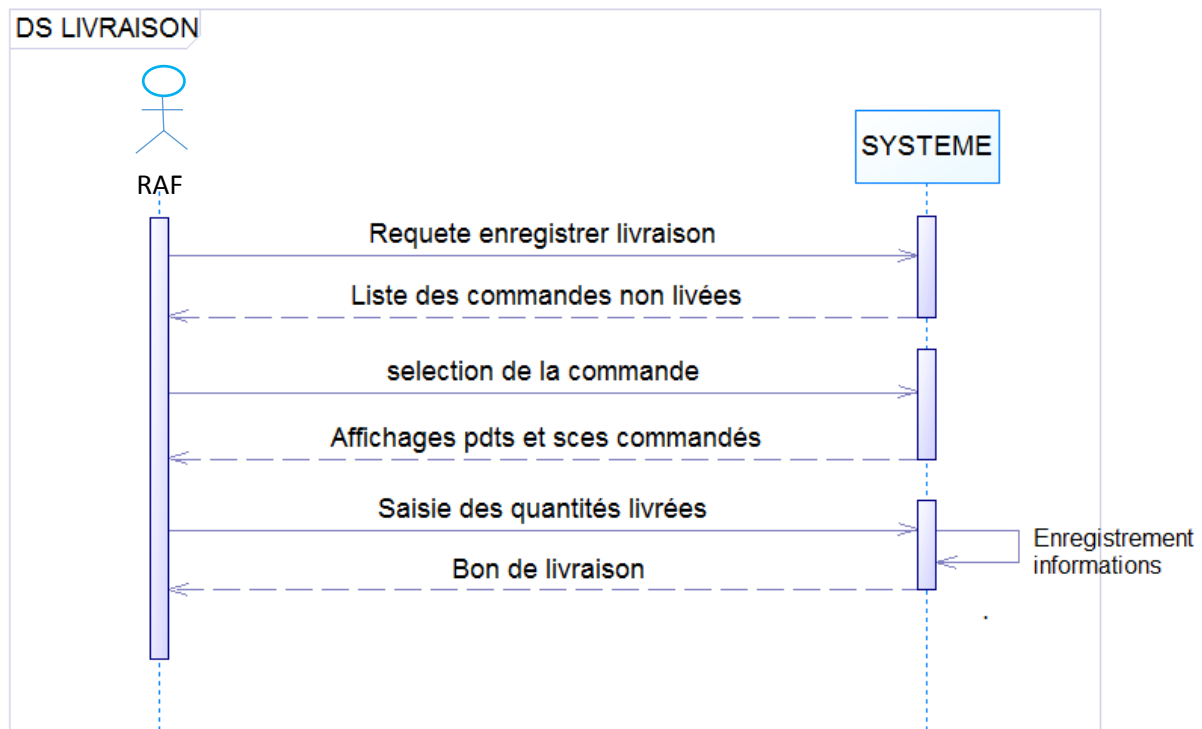


Figure 20: Diagramme de séquence, enregistrer livraison client

Cas d'utilisation « Enregistrer règlement client »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer règlement client

Résumé : Ce cas d'utilisation permet d'enregistrer le règlement d'une facture par un client

Date : 04/12/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des commandes clients non réglées
2. L'utilisateur sélectionne la commande
3. L'utilisateur sélectionne le mode de paiement et le compte à débiter
4. L'utilisateur saisit le montant du règlement et valide (A1)
5. Le système enregistre le règlement et informe l'utilisateur
6. L'utilisateur imprime le reçu de paiement

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 3 du scénario nominal

Post condition

Règlement enregistré

Diagramme de séquence « Enregistrer règlement client »

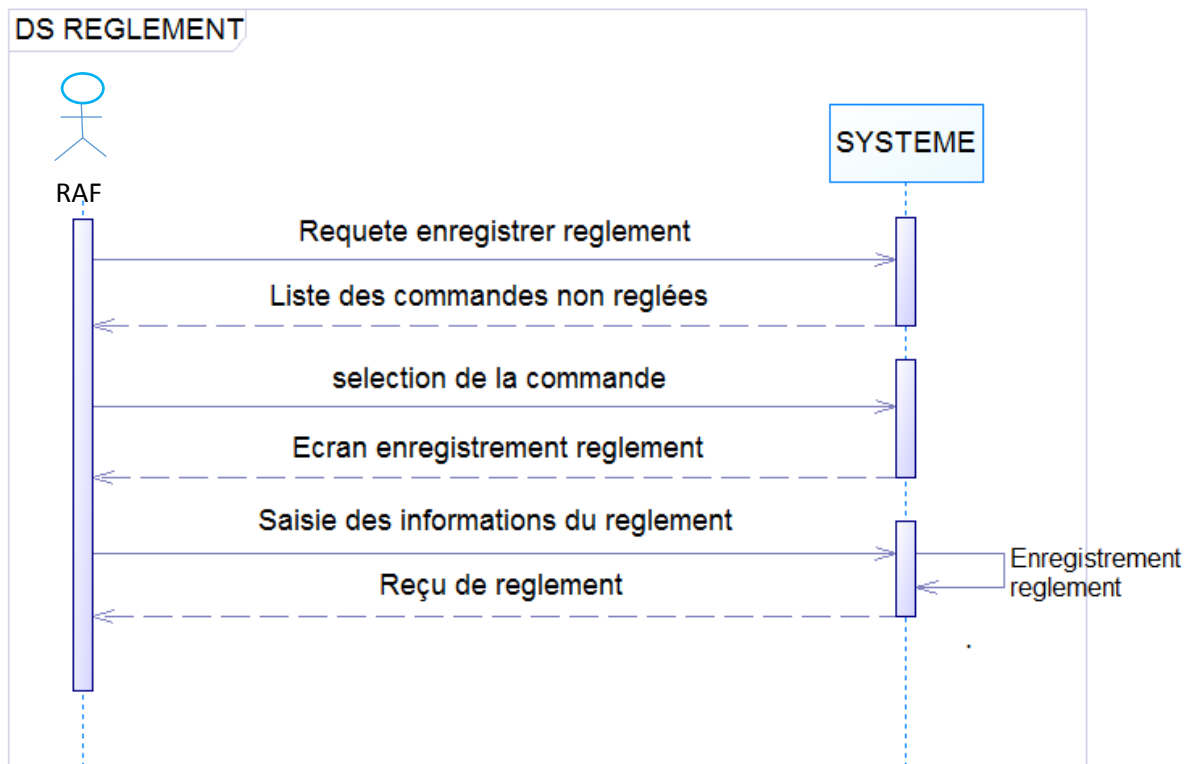


Figure 21: Diagramme de séquence, enregistrer règlement client

Diagramme de classe Vente

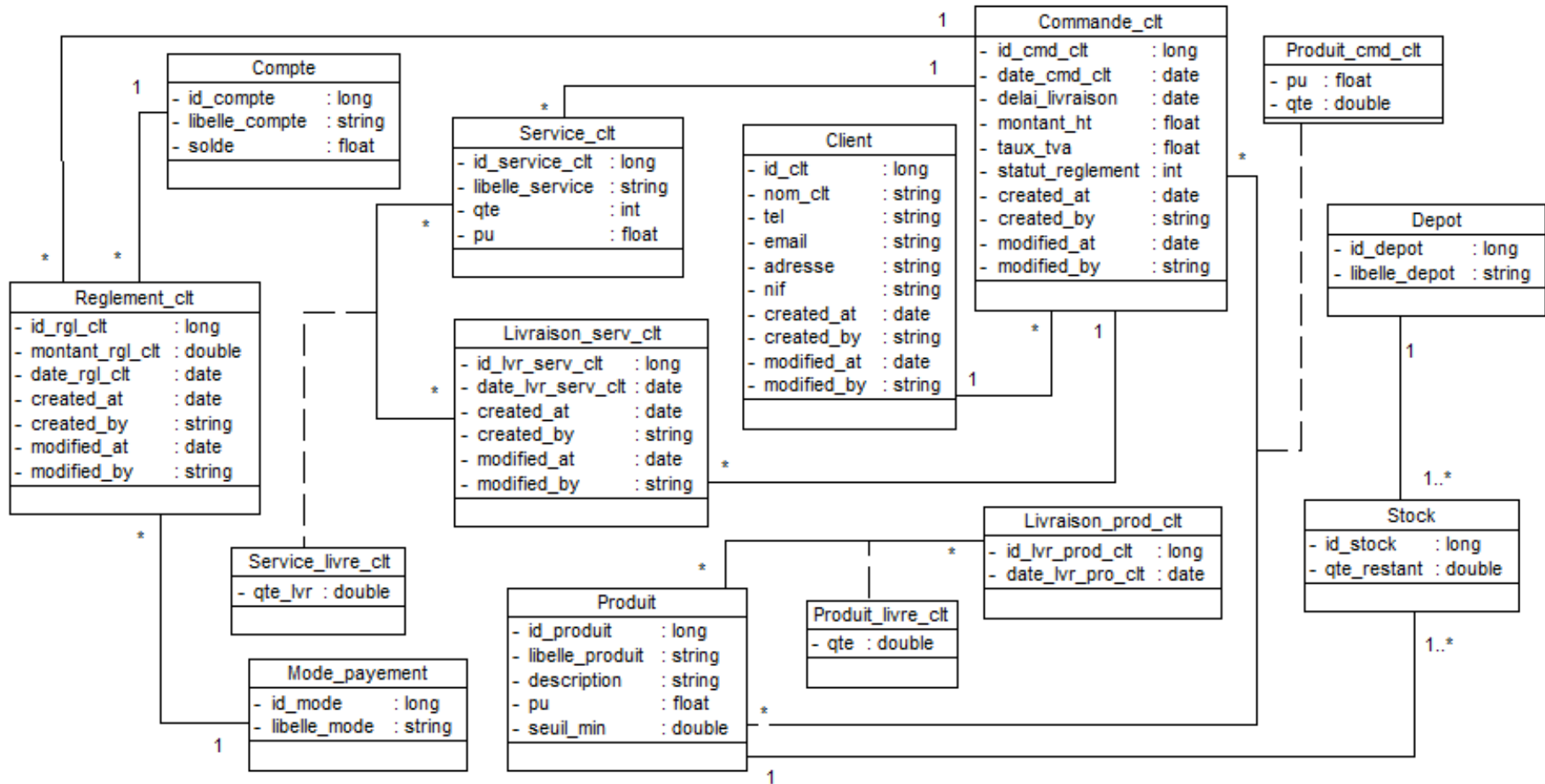


Figure 22: Diagramme de classe, gestion des ventes

IV.2.7 Sprint 5 : Gestion des tâches

Thème	User story
Gestion des tâches	En tant que RAF, je voudrais : Ajouter / consulter / modifier / supprimer / Clore une tâche
	En tant que RAF, je voudrais : Imputer/Affecter une tâche
	En tant qu'Employé, je voudrais : Affecter/ Annoter une tâche
	En tant que RAF, je voudrais : Consulter les tâches hors délais

Tableau 6: Sprint backlog, gestion des tâches

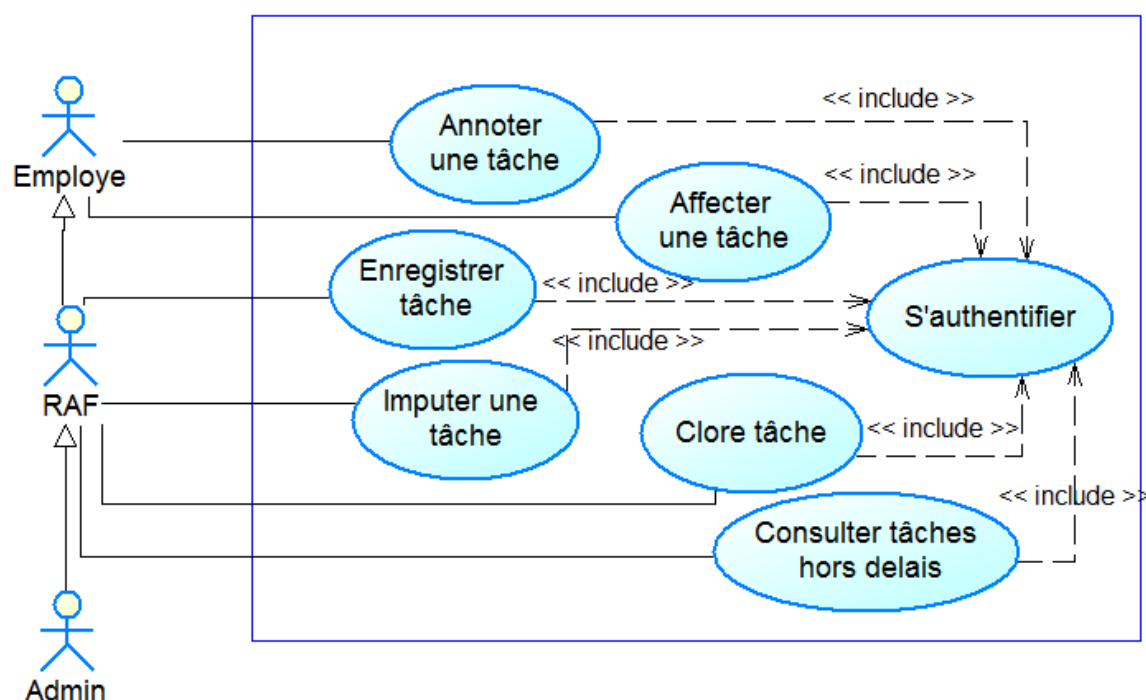


Figure 23: Diagramme des cas d'utilisation, gestion des tâches

Cas d'utilisation « Enregistrer Tâche »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Enregistrer une tâche

Résumé : Ce cas d'utilisation permet d'enregistrer une tâche

Date : 10/12/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système demande à l'utilisateur de fournir les informations concernant la tâche

2. L'utilisateur fournit les informations et valide (A1, A2)
3. Le système enregistre la tâche et informe l'utilisateur

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

A2. Date de début > Date de fin

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

Post condition

Tâche enregistrée

Cas d'utilisation « Affecter une Tâche »

Sommaire d'identification

Acteur(s) : Employé, RAF, Administrateur

Titre : Affecter une tâche

Résumé : Ce cas d'utilisation permet d'affecter une tâche à un employé

Date : 10/12/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des tâches imputées non terminées
2. L'utilisateur sélectionne la tâche et saisit les informations d'affectation (l'employé, la description du travail à effectuer) (A1)
3. Le système enregistre l'affectation et informe l'utilisateur

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

Post condition

Tâche affectée

Cas d'utilisation « Imputer une Tâche »

Sommaire d'identification

Acteur(s) : RAF, Administrateur

Titre : Imputer une tâche

Résumé : Ce cas d'utilisation permet d'imputer une tâche à un service

Date : 10/12/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des tâches non imputées
2. L'utilisateur sélectionne la tâche et saisit les informations d'imputation de la tâche (le service, la description du travail à effectuer) (A1)
3. Le système enregistre l'imputation de la tâche et informe l'utilisateur

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

Post condition

Tâche imputée

Cas d'utilisation « Annoter une Tâche »

Sommaire d'identification

Acteur(s) : Employé

Titre : Annoter une tâche

Résumé : Ce cas d'utilisation permet à un employé d'annoter une tâche

Date : 10/12/2015

Version : 0.1

Pré condition

Utilisateur authentifié

Description des enchainements

Scénario nominal

1. Le système affiche la liste des tâches non terminées, affectées à l'employé
2. L'employé sélectionne la tâche et fait son annotation (A1)
3. Le système enregistre l'annotation de la tâche et informe l'utilisateur

Scénario alternatif

A1. Erreur dans la saisie des informations

1. Le système informe l'utilisateur de la situation
2. Retour au point 2 du scénario nominal

Post condition

Tâche annotée

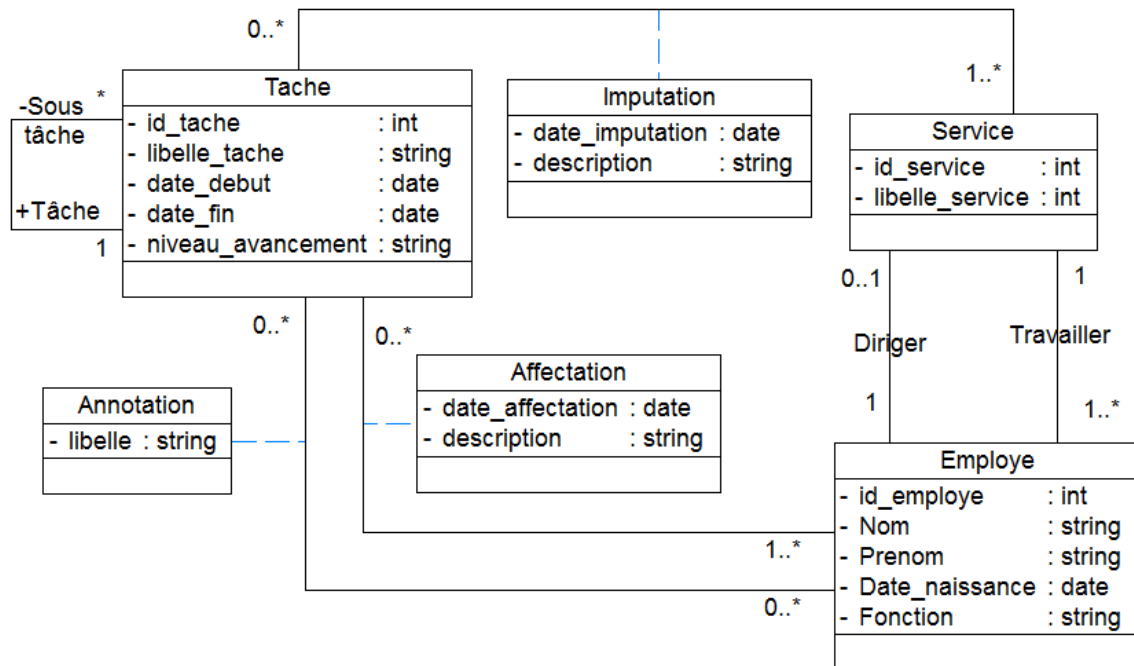


Figure 24: Diagramme de classe, gestion des tâches

Conclusion

Ce chapitre nous a permis de présenter l'analyse et la conception de notre progiciel. Après une étude de l'existant, nous avons procédé à une présentation itérative du travail effectué dans cette phase du projet.

TROISIEME PARTIE : REALISATION ET CONDUITE DE PROJET

Cette partie a pour objectif de présenter la réalisation et la conduite de notre projet. Nous y présenterons les outils de modélisation et les outils de développement choisis pour la réalisation de notre progiciel de gestion intégré. Nous y présenterons également l'architecture physique de notre système, les langages de programmation et enfin la conduite de projet.

Chapitre V : REALISATION

Introduction

Dans ce chapitre, nous allons présenter les différents outils qui nous ont permis de réaliser notre ERP. Ensuite, nous allons montrer l'architecture réseau de notre solution. Et enfin, nous allons présenter quelques captures d'écrans.

V.1 Les outils utilisés

Power AMC



PowerAMC est un logiciel de conception créé par la société *SDP*, qui permet de modéliser les traitements informatiques et leurs bases de données associées. Créé par *SDP* sous le nom *AMC*Designor*, racheté par *Powersoft*, ce logiciel est produit par *Sybase* depuis le rachat par cet éditeur en 1995. Hors de France, la version internationale est commercialisée par *Sybase* sous la marque *PowerDesigner*. PowerAMC permet de réaliser tous les types de modèles informatiques. (Source

Wikipédia)

Notepad++



Notepad++ est un éditeur de code source qui prend en charge plusieurs langages. Ce programme, codé en C++ avec STL et win32 api, a pour vocation de fournir un éditeur de code source de taille réduite mais très performant, en optimisant de nombreuses fonctions tout en conservant une facilité d'utilisation et une certaine convivialité.

(Source <http://notepad-plus-plus.org/fr/>)

Git



Git est un logiciel de gestion de versions décentralisé. C'est un logiciel libre créé par *Linus Torvalds*, auteur du noyau *Linux*, et distribué selon les termes de la licence publique générale *GNU* version 2.

Git dispose notamment des commandes suivantes (pour une liste complète, consultez la page de manuel *Git*) :

- *git init* : crée un nouveau dépôt
- *git clone* : clone un dépôt distant ;
- *git add* : ajoute de nouveaux objets blobs dans la base des objets pour chaque fichier modifié depuis le dernier commit. Les objets précédents restent inchangés ;

- git commit : intègre la somme de contrôle SHA-1 d'un objet tree et les sommes de contrôle des objets commits parents pour créer un nouvel objet commit ;
 - git branch : crée une nouvelle branche de développement ;
 - git merge : fusionne plusieurs branches de développement.
- (Source Wikipédia)

WampServer



WampServer est une plate-forme de développement Web sous Windows pour des applications Web dynamiques à l'aide du serveur Apache, du langage de scripts PHP et d'une base de données MySQL. Il possède également PHPMyAdmin permettant de gérer plus facilement les bases de données.

MySQL



MySQL est un système de gestion de base de données (SGBD). Selon le type d'application, sa licence est libre ou propriétaire. Il fait partie des logiciels de gestion de base de données les plus utilisés au monde, autant par le grand public (applications web principalement) que par des professionnels.

PHP



PHP est un langage interprété (un langage de script) exécuté du côté serveur (comme les scripts CGI, ASP, ...) et non du côté client (un script écrit en JavaScript ou une applet Java s'exécutant sur votre ordinateur...). La syntaxe du langage provient de celles du langage C, du Perl et de Java. Le PHP permet un interfaçage simple avec de nombreux systèmes de gestion de bases de données (SGBD), parmi lesquels : Adabas D, dBase, Empress, FilePro, Informix, Interbase, mSQL, MySQL, Oracle, PostgreSQL, Solid, Sybase, Velocis, Unix dbm.

Twitter Bootstrap



Twitter Bootstrap est une collection d'outils utile à la création de sites web et applications web. C'est un ensemble qui contient des codes HTML et CSS, des formulaires, boutons, outils de navigation et autres éléments interactifs, ainsi que des extensions JavaScript en option.

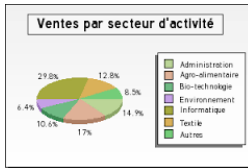
(Source : Wikipédia)

FPHP



FPDF est une classe PHP libre qui permet de créer des fichiers PDF directement depuis PHP sans dépendre d'une librairie extérieure. Le projet, commencé en 2001 par Olivier Plathey, en est aujourd'hui à la version 1.8 en date du 29 novembre 2015. (Source : Wikipédia)

Artichow



Artichow est une librairie qui permet de créer simplement des graphiques avec PHP et GD. On pourra avec cette librairie générer des courbes, des histogrammes ou encore des camemberts. Artichow supporte plusieurs types de dégradés, gère la transparence des couleurs, permet l'ajout de légendes et la personnalisation des graphes.

Adobe Photoshop



Photoshop est un logiciel de retouche, de traitement et de dessin assisté par ordinateur édité par Adobe. Il est principalement utilisé pour le traitement de photographies numériques, mais sert également à la création d'images *ex nihilo*. Il travaille essentiellement sur images matricielles car les images sont constituées d'une grille de points appelés pixels. L'intérêt de ces images est de reproduire des graduations subtiles de couleurs.

C'est le format de données conçu pour représenter les pages web. C'est un langage de balisage permettant d'écrire de l'hypertexte, d'où son nom. HTML permet également de structurer sémantiquement et de mettre en forme le contenu des pages, d'inclure des ressources multimédias dont des images, des formulaires de saisie, et des programmes informatiques. Il permet de créer des documents interopérables avec des équipements très variés de manière conforme aux exigences de l'accessibilité du web. Il est souvent utilisé conjointement avec des langages de programmation (JavaScript) et des formats de présentation (feuilles de style en cascade). HTML est initialement dérivé du Standard Generalized Markup Language (SGML). Dans le langage courant, HTML5 dernière révision majeure de HTML désigne souvent un ensemble de technologies Web (HTML5, CSS3 et JavaScript) permettant notamment le développement d'applications

V.2 Déploiement de la plateforme web

Compte tenu des besoins techniques, nous avons opté pour une architecture client/serveur à trois niveaux, avec un client léger. Dans cette architecture, les applications au niveau serveur sont délocalisées, c'est-à-dire que chaque serveur est spécialisé dans une tâche. Ainsi, on aura un serveur d'application et un serveur de base de données.

Cette architecture offre :

- Une plus grande flexibilité du système
- Une plus grande sécurité des services
- Une meilleure performance par le partage des tâches coté serveur



Figure 25: Architecture 3 tiers

V.3 Architecture réseau de la solution

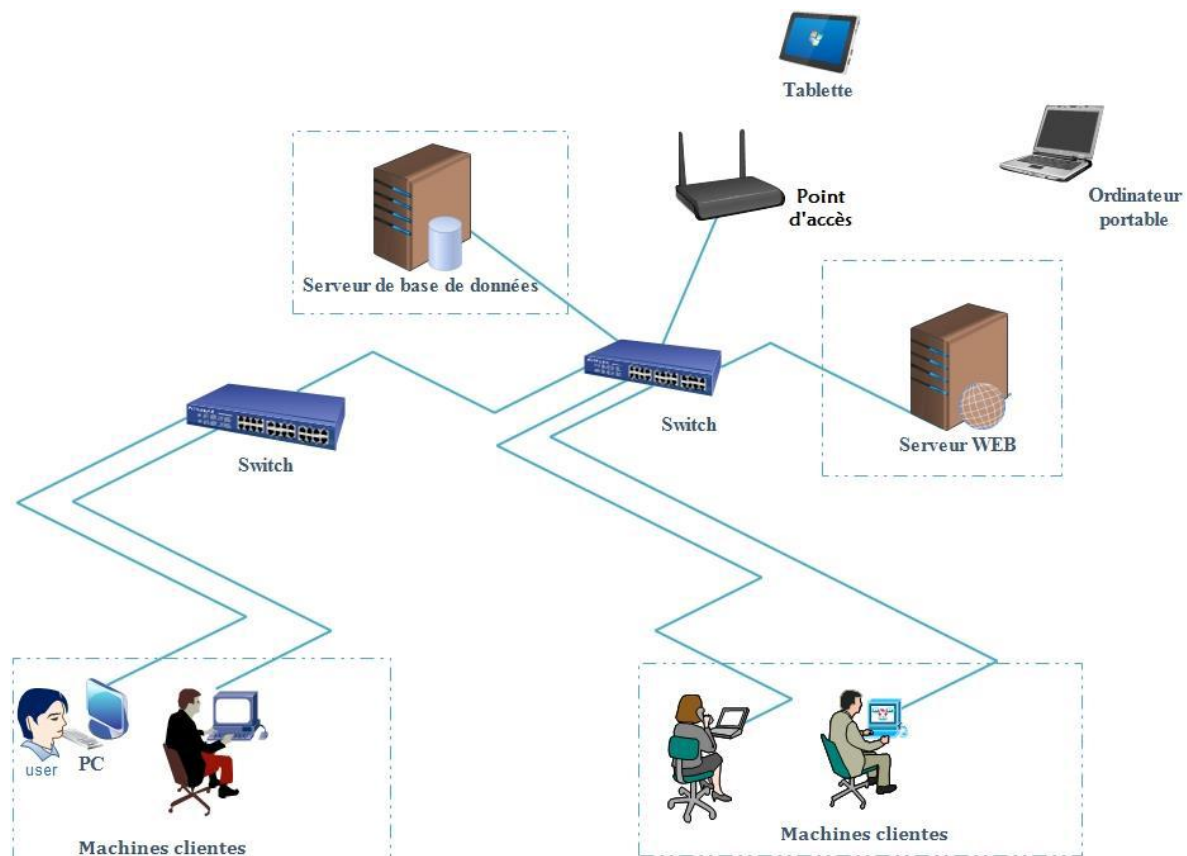


Figure 26: Architecture réseau de la solution

V.4 Menaces et risques applicatifs

A. Types d'attaques

Le WASC (Web Application Security Consortium) établie dans son rapport « WASC Threat Classification » une liste exhaustive des menaces qui pèsent sur la sécurité des applications Web. Elles sont regroupées dans six catégories définies dans la version 2004 de ce rapport.

La catégorie « authentification » regroupe les attaques de sites Web dont la cible est le système de validation de l'identité d'un utilisateur, d'un service ou d'une application.

La catégorie « autorisation » couvre l'ensemble des attaques de sites Web dont la cible est le système de vérification des droits d'un utilisateur, d'un service ou d'une application pour effectuer une action dans l'application.

La catégorie « attaques côté client » rassemble les attaques visant l'utilisateur pendant qu'il utilise l'application.

La catégorie « exécution de commandes » englobe toutes les attaques qui permettent d'exécuter des commandes sur un des composants de l'architecture du site Web.

La catégorie « révélation d'informations » définit l'ensemble des attaques permettant de découvrir des informations ou des fonctionnalités cachées.

La catégorie « attaques logiques » caractérise les attaques qui utilisent les processus applicatifs (système de changement de mot de passe, système de création de compte, ...) à des fins hostiles⁽⁵⁾.

B. Risques de sécurité

Contrairement au WASC qui décrit toutes les attaques possibles sur une application Web, l'OWASP (L'Open Web Application Security Project) ne traite que les dix plus grands risques de sécurité. Le rapport « OWASP Top 10 » permet ainsi à l'équipe projet de se focaliser sur la protection de l'application Web face aux menaces les plus importantes, ce qui est moins coûteux et plus facilement réalisable que d'essayer de se protéger de tous les dangers. L'OWASP établit le classement 2010 ci-dessous, dont chacune des failles est développée dans les chapitres suivants :

Une faille d'injection se produit quand une donnée non fiable est envoyée à un interpréteur en tant qu'élément d'une commande ou d'une requête. Les données hostiles de l'attaquant peuvent duper l'interpréteur afin de l'amener à exécuter des commandes inhabituelles ou accéder à des données non autorisées.

Les failles de Cross-Site Scripting (XSS) se produisent chaque fois qu'une application prend des données non fiables et les envoie à un navigateur Web sans validation. XSS permet à des attaquants d'exécuter du script dans le navigateur de la victime afin de détourner des sessions utilisateur, défigurer des sites web, ou rediriger l'utilisateur vers des sites malveillants.

Les failles de violation de gestion d'authentification et de session se produisent quand les fonctions correspondantes ne sont pas mises en œuvre correctement, permettant aux attaquants de compromettre les mots de passe, clés, jetons de session, ou d'exploiter d'autres failles d'implémentation pour s'approprier les identités d'autres utilisateurs.

Une faille de référence directe à un objet se produit quand un développeur expose une référence à une variable interne, tel un nom de fichier, de dossier, un enregistrement de base de données, ou une clé de base de données. Sans un contrôle d'accès ou autre protection, les attaquants peuvent manipuler ces références pour accéder à des données non autorisées.

Une attaque par falsification de requête inter-sites (CSRF) force le navigateur d'une victime authentifiée à envoyer une requête http, comprenant le cookie de session de la victime

ainsi que toute autre information automatiquement incluse, à une application web vulnérable. Ceci permet à l'attaquant de forcer le navigateur de la victime à générer des requêtes, l'application vulnérable considérant alors qu'elles émanent légitimement de la victime.

Une faille due à une mauvaise configuration de sécurité se produit quand les serveurs d'application, serveurs web, serveur de base de données, et la plate-forme n'ont pas de configuration sécurisée correctement établie et déployée. Tous ces paramètres doivent être définis, mis en œuvre, et maintenu. Ceci implique de maintenir tous les logiciels à jour, notamment toutes les bibliothèques de code employées par l'application.

Une faille de stockage de données non sécurisées se produit quand une application Web ne protège pas correctement les données sensibles, telles que les numéros de cartes de crédit, de sécurité sociale, les informations d'authentification, avec un algorithme de chiffrement ou de hash approprié. Les pirates peuvent voler ou modifier ces données faiblement protégées pour perpétrer un vol d'identité et d'autres crimes, tels que la fraude à la carte de crédit.

La défaillance dans la restriction des accès à une URL se produit quand une application web ne protège pas l'accès aux URL. Les applications doivent effectuer des contrôles d'accès similaires chaque fois que ces pages sont accédées, sinon les attaquants seront en mesure de forger des URL pour accéder à ces pages cachées.

La faille de protection de la couche transport se produit quand les applications ne peuvent pas chiffrer et protéger la confidentialité et l'intégrité du trafic réseau sensible. De plus, quand elles le font, elles supportent parfois des algorithmes faibles, utilisent des certificats expirés ou invalides, ou ne les emploient pas correctement.

Une faille de redirection et renvoi non validés se produit quand une application Web réoriente les utilisateurs vers d'autres pages et sites web, et utilise des données non fiables pour déterminer les pages de destination. Sans validation appropriée, les attaquants peuvent rediriger les victimes vers des sites de phishing ou de logiciel malveillant, ou utiliser les renvois pour accéder à des pages non autorisées⁽⁵⁾.

C. Correspondances entre les définitions de l'OWASP et du WASC

Le tableau ci-dessous fait le lien entre les catégories et les attaques définies par le WASC dans le rapport de 2010 et les menaces identifiées par l'OWASP dans le classement de 2010.

Risques identifiés par l'OWASP en 2010	Attaques identifiées par le WASC en 2010	Catégories
Injection	SQL Injection (WASC-19) XML Injection (WASC-23) Null Byte Injection (WASC-28) LDAP Injection (WASC-29) Mail Command Injection (WASC-30) OS Commanding (WASC-31) XPath Injection (WASC-39) XQuery Injection (WASC-46)	Exécution de commandes
Cross-Site Scripting	Cross-Site Scripting (WASC-08)	Attaques côté client
Violation de Gestion d'Authentification et de Session	Insufficient Authentication (WASC-01) Brute Force (WASC-11) Credential/Session Prediction (WASC-18) Session Fixation (WASC-37) Insufficient Session Expiration (WASC-47)	Autorisation, Authentification

Référence directe à un objet non sécurisée	Insufficient Authentication (WASC-01) Insufficient Authorization (WASC-02) Path Traversal (WASC-33) Predictable Location (WASC-34)	Autorisation, Authentification, Révélation d'informations
Falsification de requêtes inter-sites	Cross-Site Request Forgery (WASC-09)	Attaques logiques
Gestion de configuration non sécurisée	Server Misconfiguration (WASC-14) Application Misconfiguration (WASC-15)	Révélations d'informations
Stockage de données non sécurisé	Insufficient Data Protection (WASC-50)	Révélations d'informations
Défaillance de restriction d'accès à une URL	Insufficient Authorization (WASC-02) Denial of Service (WASC-10) Brute Force (WASC-11) Insufficient Anti-automation (WASC-21) Predictable Location (WASC-34)	Autorisation, Authentification, Révélation d'informations
Communications non sécurisées	Insufficient Transport Layer Protection (WASC-04)	Révélations d'informations
Redirection et renvoi non validés	URL Redirector Abuse (WASC-38)	Attaques logiques

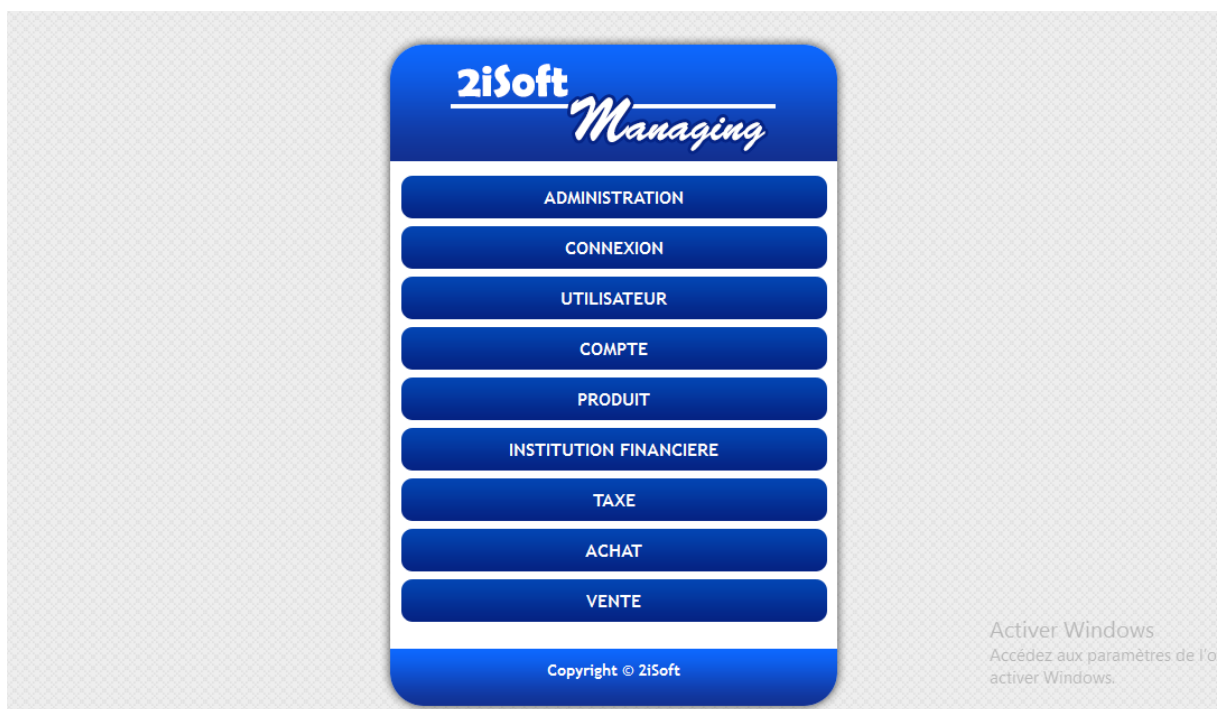
Tableau 7: Correspondances entre les définitions de l'OWASP et du WASC

V.5 Captures d'écran

Ecran de connexion



Afficher module



Enregistrer une vente

Nouvelle Vente

Commande

Client :

Date :

TVA :

Delais livraison :

saminou

09-12-2015

oui

21-10-2015

Produit

Produit :

Prix unitaire :

Quantité :

Ecran

30000

4

Ajouter

Service

Rubrique :

Prix :

Quantité :

logo

12000

1

Ajouter

Désignation	Prix Unitaire	Quantité	Sous-total	Action
1	56000	6	336000	Supprimer
site web	45000	2	90000	Supprimer

Montant total: 426000

Valider

Enregistrer une livraison

2iSoft Managing

Basculer vers

Log out

GESTION DES ACHATS

Niamey, le 09/12/2015. Il est 14:33:38

PLACEZ ICI LE CONTENU DE CLASS "INFO"

Créer Client Liste Clients Créer Vente Livraison Règlement Facturation

Livraison

Recherche

Client	Date commande	Delai livraison	Montant	Action
saminou	2015-11-04 00:00:00	2000-12-12	17845500	

N° commande: 1 Date: 2015-11-04 00:00:00

Client: saminou

Désignation	Prix unitaire	Quantité	Sous-total	Quantité reçu	Livraison
Ordinateur	767676	23	17656548	0	
Ecran	4545	12	54540	0	
Logo	546	6	3276	0	
site web	65567	2	131134	0	

Enregistrer livraison

Enregistrer un règlement

[Créer Client](#) [Liste Clients](#) [Créer Vente](#) [Livraison](#) [Règlement](#) [Facturation](#)

Règlement

▼ Recherche

Client	Date commande	Delai livraison	Montant	Action
saminou	2015-11-04 00:00:00	2000-12-12	17845500	✕

N° commande: 1 Date: 2015-11-04 00:00:00
Client: saminou

Désignation	Prix unitaire	Quantité	Sous-total	Quantité reçu
Ordinateur	767676	23	17656548	0
Ecran	4545	12	54540	0
Logo	546	6	3276	0
site web	65567	2	131134	0

Montant total HT17845498

Montant TVA0.12

Montant TTC19986957.76

Mode paiement▼

Montant déjà versé2000

Compte à débiter▼

Reste à payer19984957.76

Nouveau versement

valider

Activer Windows
Accédez aux paramètres de l'ordinateur pour activer Windows.

Copyright © 2iSoft

Conclusion

Ce chapitre nous a permis de présenter la mise en œuvre de notre progiciel de gestion intégré. Cette réalisation a été possible grâce aux différents outils de développement utilisés.

Chapitre VI : Conduite de projet

Introduction

Un projet est un processus unique qui consiste en un ensemble d'activités coordonnées et maîtrisées, comportant des dates de début et de fin, entrepris dans le but d'atteindre un objectif conforme à des exigences spécifiques, incluant des contraintes de délai, de coût et de ressource ; il fait donc l'objet d'une budgétisation de moyens et d'un bilan indépendant de celui de l'entreprise. La gestion d'un projet devient alors la démarche visant à organiser de bout en bout le bon déroulement du projet, coordonner, diriger et surtout contrôler les ressources utilisées et ce, de façon structurée et optimale. Nous présenterons dans cette dernière partie du document l'organisation de notre projet notamment, les intervenants au projet, les estimations des ressources humaines, matérielles et financières. Nous terminerons par le bilan et les perspectives du travail effectué, sans bien sûr oublier de noter les apports et difficultés liés à notre stage.

VI.1 Intervenants au projet

Au cours de notre stage, nous avons travaillé avec plusieurs personnes dont le responsable du projet, notre encadreur et notre superviseur académique :

Intervenant	Titre	Rôle Scrum
HAMZA ISSA Faroukou	Directeur Général de 2isoft (Maître de stage)	Product owner et Membre de l'équipe de développement
SEYDOU Zourkaleini	Responsable formation à 2isoft	Scrum master
ALI TASSIOU Abass	Etudiant Ing3 (Stagiaire)	Membre de l'équipe de développement
MAHAMAN DIALLO Abdoul Moumouni	Développeur 2isoft	Membre de l'équipe de développement
Mr MATY Maman	Enseignant à IAI(Superviseur)	-

Tableau 8: Intervenants au projet

VI.2 Estimation du cout de réalisation

Ressources utilisées	Coût
Ordinateur portable HP + Windows 8	400 000 FCFA
Environnement WAMP	Gratuit
FPDF	Gratuit
Artishow	Gratuit
Bootstrap	Gratuit
Notepad++	Gratuit
PowerAMC	Disponible
Prime de stage	100 000 FCFA (*5)
COUT TOTAL	900 000 FCFA

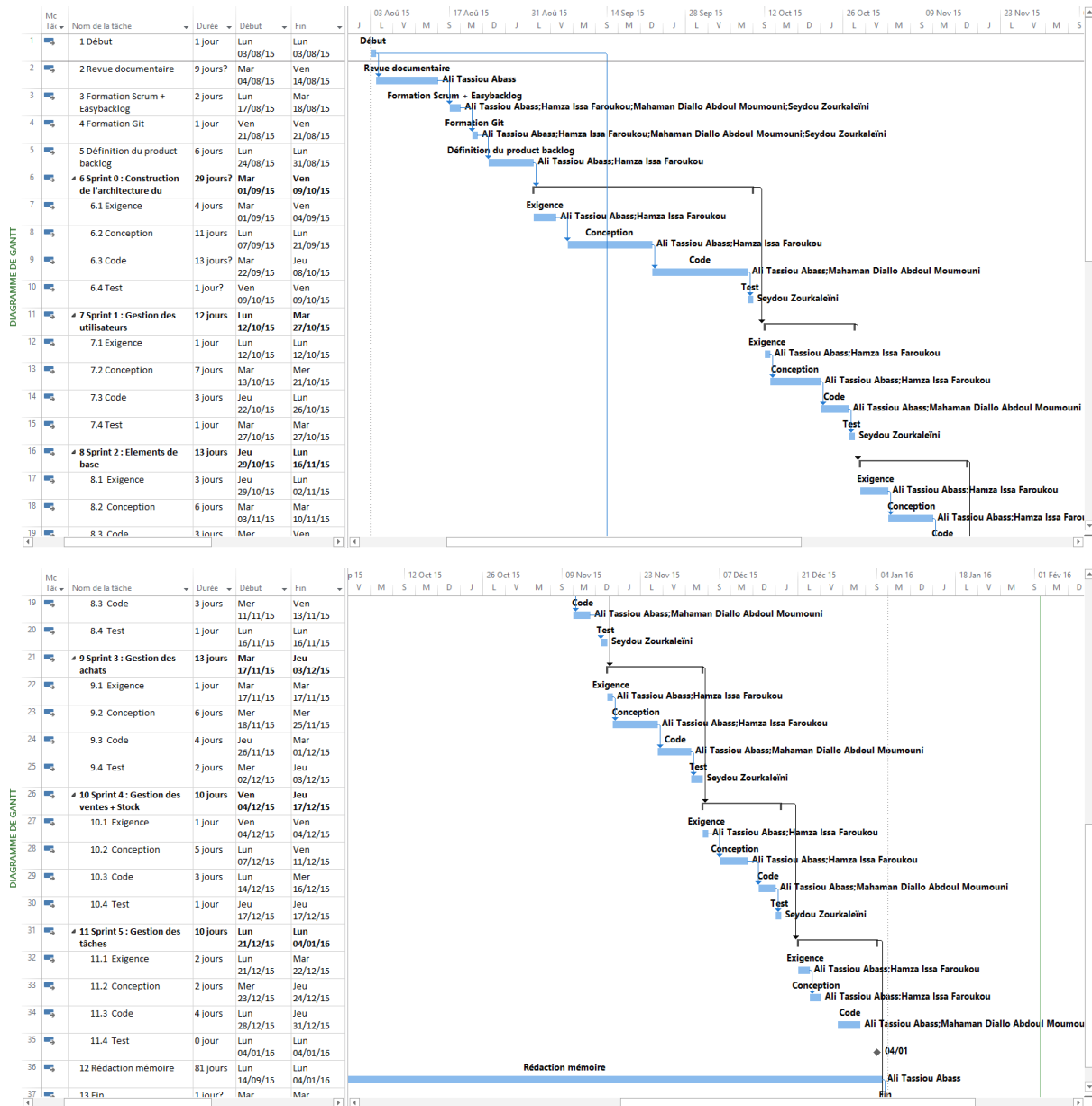
Tableau 9: coût de réalisation

VI.3 Planification des tâches

	Nom de la tâche	durée	début	fin
1	Début du projet	1 jour	03 aout 2015	03 aout 2015
2	Revue documentaire	9 jours	04 aout 2015	14 aout 2015
3	Formation Scrum + Easybacklog	2jours	17 aout 2015	18 aout 2015
4	Formation git	1 jour	21 aout 2015	21 aout 2015
5	Définition du product backlog	6 jours	24 aout 2015	31 aout 2015
Sprint 0 : construction de l'architecture du système (le cœur)				
6	exigence	4jours	01 septembre 2015	04 septembre 2015
7	conception	11 jours	07 septembre 2015	21 septembre 2015
8	code	13 jours	22 septembre 2015	08 octobre 2015
9	test	1 jour	09 octobre 2015	09 octobre 2015
Sprint 1 : gestion des utilisateurs				
10	exigence	1 jour	12 octobre 2015	12 octobre 2015
11	conception	7 jours	13 octobre 2015	21 octobre 2015
12	code	3 jours	22 octobre 2015	26 octobre 2015
13	test	1 jour	27 octobre 2015	27 octobre 2015
Sprint 2 : Eléments de base				
14	exigence	3 jours	29 octobre 2015	02 novembre 2015
15	conception	6 jours	03 novembre 2015	10 novembre 2015
16	code	3 jours	11 novembre2015	13 novembre 2015
17	test	1 jour	16 novembre2015	16 novembre 2015
Sprint 3 : gestion des achats				
18	exigence	1 jour	17 novembre 2015	17 novembre 2015
19	conception	6 jours	18 novembre 2015	25 novembre 2015
20	code	4 jours	26 novembre 2015	01 décembre 2015
21	test	2 jours	02 décembre 2015	03 décembre 2015
Sprint 4 : gestion des ventes + stock				
22	exigence	1 jour	04 décembre 2015	04 décembre 2015
23	conception	5 jours	07 décembre 2015	11 décembre 2015
24	code	3 jours	14 décembre 2015	16 décembre 2015
25	test	1 jour	17 décembre 2015	17 décembre 2015
Sprint 5 : gestion des tâches				
26	exigence	2 jours	21 décembre 2015	22 décembre 2015
27	conception	2 jours	23 décembre 2015	24 décembre 2015
28	code	4 jours	28 décembre 2015	31 décembre 2015
29	test	1 jour	04 janvier 2016	04 janvier 2016
30	Rédaction du mémoire	81 jours	14 septembre 2015	04 janvier 2016

Tableau 10: Planification des tâches

VI.4 Diagramme de Gantt prévisionnel

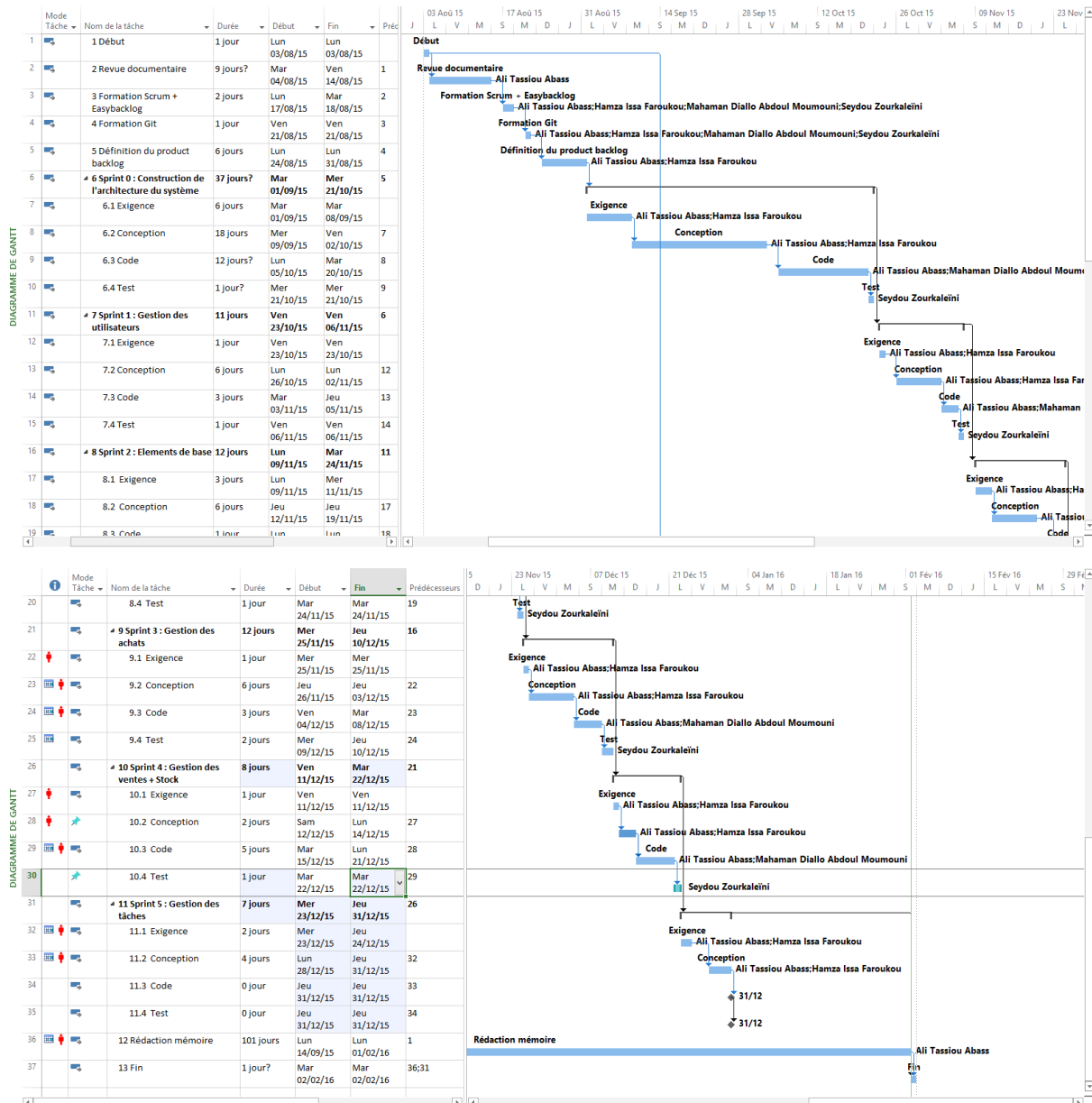


VI.5 Exécution des tâches

	Nom de la tâche	durée	début	fin
1	Début du projet	1 jour	3 aout 2015	3 aout 2015
2	Revue documentaire	9 jours	4 aout 2015	14 aout 2015
3	Formation Scrum + Easybacklog	2jours	17 aout 2015	18 aout 2015
4	Formation git	1 jour	21 aout 2015	21 aout 2015
5	Définition du product backlog	6 jours	24 aout 2015	31 aout 2015
Sprint 0 : construction de l'architecture du système (le cœur)				
6	exigence	6jours	1 septembre 2015	8 septembre 2015
7	conception	18 jours	9 septembre	2 octobre 2015
8	code	12 jours	5 octobre 2015	20 octobre 2015
9	test	1 jour	21 octobre 2015	21 octobre 2015
Sprint 1 : gestion des utilisateurs				
10	exigence	1 jour	23 octobre 2015	23 octobre 2015
11	conception	6 jours	26 octobre 2015	2 novembre2015
12	code	3 jours	3 novembre2015	5 novembre2015
13	test	1 jour	6 novembre2015	6 novembre2015
Sprint 2 : Eléments de base				
14	exigence	3 jours	9 novembre2015	11 novembre2015
15	conception	6 jours	12 novembre2015	19 novembre2015
16	code	2 jours	20 novembre2015	23 novembre2015
17	test	1 jour	24 novembre2015	24 novembre2015
Sprint 3 : gestion des achats				
18	exigence	1 jour	25 novembre2015	25 novembre2015
19	conception	6 jours	26 novembre2015	3 décembre 2015
20	code	3 jours	4 décembre 2015	8 décembre 2015
21	test	2 jours	9 décembre 2015	10 décembre 2015
Sprint 4 : gestion des ventes + stock				
22	exigence	1 jour	11 décembre 2015	11 décembre 2015
23	conception	2 jours	12 décembre 2015	14 décembre 2015
24	code	5 jours	15 décembre 2015	21 décembre 2015
25	test	1 jour	22 décembre 2015	22 décembre 2015
Sprint 5 : gestion des tâches				
26	exigence	2 jours	23 décembre 2015	24 décembre 2015
27	conception	4 jours	28 décembre 2015	31 décembre 2015
28	code	0 jour		
29	test	0 jour		
30	Rédaction du mémoire	90 jours	14 septembre 2015	01 février 2016

Tableau 11: Exécution des tâches

VI.6 Diagramme de Gantt réel



Conclusion

Ce chapitre nous a permis de présenter la conduite de notre projet, à savoir les intervenants au projet, la planification des tâches et l'estimation du coût de réalisation.

Conclusion générale

Notre travail a consisté à automatiser la gestion interne de 2isoft au travers d'un ERP. Après analyse des avantages et inconvénients des différentes alternatives, sans oublier les contraintes fixées par 2isoft, notre choix s'est porté sur un développement sur mesure, fait par des agents de 2isoft. Cette solution a permis de gérer les achats et les ventes des produits et services ; gérer le paiement des taxes et cotisations aux établissements financiers ; gérer la trésorerie ; éditer les bons de commandes ; enregistrer les livraisons ; gérer les tâches ; gérer les employés ; gérer les clients et les fournisseurs. L'application permettra ainsi d'augmenter la productivité de l'entreprise et éviter le problème de fragmentation de l'information.

Nous avons abordé l'étude par une définition des contours du sujet et des concepts qui lui sont liés. Nous avons ensuite dressé l'analyse et la conception du système, puis la mise en œuvre pratique de la solution.

Ce travail nous a permis de mettre en application les connaissances théoriques, mais aussi pratiques acquises au cours de nos années de formation ; d'approfondir nos connaissances sur les méthodes agiles en générale et Scrum en particulier ; d'avoir une expérience en matière de gestion des projets informatiques ; de renforcer notre capacité à travailler en équipe.

Toutefois, nous avons rencontré quelques difficultés durant ce stage. Il s'agit notamment du fonctionnement modulaire de notre application ; de l'utilisation de git et Scrum ; du contrôle d'accès aux fonctionnalités de l'application.

Au stade actuel, le résultat obtenu est satisfaisant. La version beta de l'application est déjà en exploitation. Néanmoins, il nous reste à développer le module sur la gestion des ressources humaines et terminer l'implémentation de la gestion des tâches.

Comme perspective, nous notons déjà que l'architecture de notre application sera réutilisée dans les futurs développements de 2isoft ; par ailleurs le progiciel que nous avons développé sera proposé aux clients de 2isoft pour gérer leurs activités.

REFERENCES BIBLIOGRAPHIQUES

Ouvrage

- [1] Sophie MOURLON et Laurent NEYER, Mémoire d'Ingénieurs, Corps Techniques de l'Etat, Ecole des mines de Paris : Tout ce que nous avons voulu savoir sur les ERP, 2002
- [4] Ken Schwaber et Jeff Sutherland, Le Guide Scrum, Juillet 2013
- [5] Guillaume HARRY, Failles de sécurité des applications Web, 03/04/2012
- [6] PASCAL (ROQUES), FRANK VALLET, UML2 en action, Eyrolles, Paris, 2007, 394 Pages ;

Sites web consultés

- [2]<http://www.agi-paris.fr/lecteur-de-faq/items/quelles-sont-les-avantages-et-les-inconvenients-dun-erp-.html>
- [3]<http://fablain.developpez.com/tutoriel/presenterp/> , Présentation générale des ERP